COLLEGE OF ENGINEERING TRIVANDRUM

Module 2

8051 Programming

Assembly · Embedded C · I/O
Timers · Counters

PCEOT501
Microcontrollers and Embedded Systems
S5 Electrical and Computer Engineering

A complete examination companion

Official 11-hour Module 2 syllabus; all seven classroom decks reviewed (269 physical pages). Corrected, selectable programs; independent calculations; answered practice; source and verification records.

Prepared 7 October 2026
Classic 12-clock 8051 baseline · Keil A51 / C51 notation

Contents

1 Scope, assumptions and study route 4
1.1 Syllabus coverage map . . . 4
1.2 How to prepare . . . 4

2 Essential architecture and memory spaces 5
2.1 Registers used in this module . . . 5
2.2 The memory map . . . 5
2.3 Byte address versus bit address . . . 6

3 Assembly source, data representation and directives 7
3.1 An assembly statement . . . 7
3.2 From source to execution . . . 7
3.3 Numbers, bytes, signed values, BCD and ASCII . . . 7
3.4 Assembler directives . . . 8
3.5 Instruction bytes are not instruction cycles . . . 9

4 Addressing modes and operand reasoning 10
4.1 A worked value/address trace . . . 10

5 Complete data-transfer programs 11
5.1 A01: Immediate, register, direct and indirect transfers . . . 11
5.2 A02: CODE table lookup using MOVC . . . 11
5.3 A03: XDATA write and readback using MOVX . . . 12
5.4 A04: Copy 17 bytes to external memory . . . 12
5.5 A05: Exchange two nonoverlapping five-byte blocks . . . 13

6 Arithmetic, PSW flags and execution traces 14
6.1 PSW and flag rules . . . 14
6.2 How to calculate flags . . . 14
6.3 A06: Classroom addition of 25H, 34H and 12H . . . 15
6.4 A07: An 8-bit addition with a preserved carry byte . . . 16
6.5 A08: Multi-byte addition with propagated carry . . . 16
6.6 A09: Subtraction with an explicit initial borrow . . . 17
6.7 A10: A 16-bit subtraction . . . 17
6.8 A11: Multiplication with both result bytes preserved . . . 18
6.9 A12: Division, quotient/remainder and zero divisor . . . 18
6.10 A13: Decimal adjustment and increment/decrement . . . 19

7 Supporting flow control, calls, stack and software delay 20
7.1 The jumps you need . . . 20
7.2 Relative offsets and 2 KB boundaries . . . 20
7.3 Stack operation . . . 20
7.4 A14: A call with an exact stack trace . . . 21
7.5 Counted software delay and a valid whole-port toggle . . . 21
7.6 A15: Port-latch toggle with a 100-count delay . . . 22
7.7 A16: Classroom 55H/AAH output on P0 . . . 23
7.8 A17: Monitor an input and generate a logic strobe . . . 23

8 Embedded C: types, delay and I/O 24
8.1 C51 data types and ranges . . . 24
8.2 Hardware and memory extensions . . . 24
8.3 Integer promotion, masks and Boolean inversion . . . 24

8.4 Time delay in C . . . 25
8.5 Ports: latch, pin and circuit . . . 25
8.6 C01: Alternate 55H and AAH on P1 . . . 26
8.7 C02: Toggle only P2.4 . . . 27
8.8 C03: Sample P1, wait, then copy to P2 . . . 28
8.9 C04: Route a P0 input by its unsigned magnitude . . . 29
8.10 C05: Send exactly the ASCII sequence 012345ABCD . . . 29
8.11 C06: Invert input P1.0 onto P2.7 . . . 30
8.12 C07: Sensor-controlled square-wave driver signal . . . 30

9 Timer/counter hardware, registers and calculations 31
9.1 Registers and control bits . . . 31
9.2 The four modes . . . 32
9.3 Derive the timing formulas . . . 32
9.4 Recomputed classroom preloads . . . 33
9.5 Hardware interval versus complete program delay . . . 34
9.6 DS89C420 is a different timing target . . . 34

10 Complete Embedded C timer and counter programs 35
10.1 C08: Timer 0 mode-1 50 ms hardware intervals . . . 35
10.2 C09: Timer 1 mode-1 repeated 25 ms intervals . . . 36
10.3 C10: Timer 1 mode-1 50 Hz square wave . . . 37
10.4 C11: Timer 1 mode-2 auto-reload waveform . . . 37
10.5 C12: A long delay from continuous mode-2 overflows . . . 38
10.6 C13: Switch-selectable 500 / 750 Hz signal . . . 39
10.7 External event counters: required syllabus material . . . 39
10.8 C14: Timer 1 mode-2 8-bit event display . . . 40
10.9 C15: Timer 0 mode-1 coherent 16-bit event display . . . 41
10.10 C16: A deliberate stop-and-read contrast . . . 42

11 Answered examination practice 43
11.1 Short questions with concise answers . . . 43
11.2 Operand and execution-trace drills . . . 44
11.3 Developed 9-mark choices: assembly and arithmetic . . . 45
11.4 Developed 9-mark choices: Embedded C and I/O . . . 46
11.5 Developed 9-mark choices: timers . . . 46
11.6 Developed 9-mark choices: counters . . . 47
11.7 Modified calculations and debugging . . . 47

12 Final revision and reading route 49
12.1 Compact revision sheet . . . 49
12.2 Syllabus completion checklist . . . 49
12.3 Prescribed books: verified bibliography and chapter map . . . 49

13 Supporting classroom instruction material 51
13.1 Classroom signal examples with later-module peripherals . . . 52

14 Classroom discrepancies, recovery and verification 53
14.1 Discrepancy appendix . . . 53
14.2 Page-by-page source inventory and recovery map . . . 56
14.3 Primary references and access record . . . 62
14.4 Verification record and remaining limits . . . 63

1 Scope, assumptions and study route

The controlling document is the official Electrical and Computer Engineering syllabus: the course begins on physical PDF page 128; Module 2 continues onto page 129. The examination pattern is on page 130 and prescribed books on page 131. Module 2 has **11 contact hours**. Its assembly-program scope is **data transfer and arithmetic operations**. Jumps, calls and selected logical/bit instructions support those routines and I/O; their wider instruction catalogue is supplementary.

Counter programming is required. The "for information only" labels in Class 7 do not override the syllabus. Serial communication, interrupt handlers, LCD drivers, ADC/DAC and motor control are Module 3 topics. A classroom LCD strobe is retained only to explain byte data plus a control pin.

Processor, tools and circuit contract

Unless explicitly changed: original-compatible 8051, 128 bytes internal RAM, classic 12-clock machine cycle, two timers, 5 V logic, no enabled interrupts. Assembly uses **Keil A51**, `$MOD51` for standard SFR symbols, hexadecimal suffix `H`, semicolon comments and a final `END`. C uses **Keil C51**, `reg51.h`, `void main(void)` and its `sbit/bit/code` extensions. Every program is a separate application; do not combine their `main` functions.
Timing chapters normally use **11.0592 MHz**; Programs C01–C07 and C11 explicitly use **12 MHz**. Hardware intervals exclude software overhead unless stated. GPIO examples assume internal program memory and no external memory bus occupying the selected ports. MOVX programs instead assume suitable external RAM, address latch, decoding and bus connections.

1.1 Syllabus coverage map

Printed page numbers count the cover as page 1 and therefore match physical PDF pages. References below are generated from the final document labels.

Official syllabus item	Section	Page
Introduction to assembly programming	3	7
Data types and assembler directives	3.3; 3.4	7, 8
8051 addressing modes	4	10
Simple ALP: data transfer	5	11
Simple ALP: arithmetic operations	6	14
8051 C: data types	8	24
8051 C: time delay	8.4; 10	25, 35
8051 C: I/O programming	8.5	25
Embedded C: timer programming	9–10	31–35
Embedded C: counter programming	10.7	39

1.2 How to prepare

Study memory and operands first; trace every transfer and arithmetic example; then learn C and port wiring; finally derive timer counts and configure counters. Attempt the practice bank before reading its answers. Treat the correction appendix as a diagnostic aid, not an extra syllabus. The lab manual contributes arithmetic/data-transfer practice only.

The end-semester examination is **60 marks in 2 ½ hours**: Part A has eight compulsory 3-mark questions, two from each of four modules (24 marks). Part B gives two 9-mark full questions per module; answer one per module (36 marks). Each full question has at most three subdivisions. Module 2 thus contributes 6 marks in Part A and one selected 9-mark answer in Part B. Practice here is authored preparation, not predicted questions or an official marking scheme.

2 Essential architecture and memory spaces

An instruction operates on a value stored somewhere. The CPU register, memory space and addressing method determine *which* stored value it accesses. The architecture is 8-bit because its ordinary data path and arithmetic operate on bytes; this does not make every register 8-bit.

2.1 Registers used in this module

Register	Width / address	Purpose
A / ACC	8 bits / E0H	Accumulator; main arithmetic operand, MOVC/MOVX data register.
B	8 bits / F0H	Second operand and high product or remainder for MUL/DIV.
DPTR	16 bits	DPH (83H):DPL (82H); code-table base or 16-bit XDATA address.
PC	16 bits	Address of next code byte; not an ordinary byte-addressed SFR.
SP	8 bits / 81H	Last used internal stack address; reset value 07H.
PSW	8 bits / D0H	Flags and register-bank selection; bit-addressable.
R0–R7	8 bits each	Current bank's eight RAM registers; R0/R1 also serve as indirect pointers.

A pair of 8-bit SFRs forms each 16-bit timer. The CPU uses high and low bytes separately; multi-byte data needs several instructions. `A` is the special accumulator operand; `ACC` names its direct SFR address. Use `PUSH ACC`, not `PUSH A`.

2.2 The memory map

CODE: 0000H–FFFFH
Instructions and constant tables
Fetch via PC; tables via MOVC

Internal DATA: 00H–7FH
Registers, bit RAM, scratch, stack
MOV direct or @R0/@R1

XDATA: 0000H–FFFFH
External data-memory address space
MOVX via DPTR or @Ri

SFR: direct 80H–FFH
Ports, PSW, timers, control
MOV direct; selected bits accessible

The same numerical address can name different spaces. CODE 0030H, internal RAM 30H and XDATA 0030H are separate locations. A classic 8051 has 4 KB on-chip program ROM but can address 64 KB CODE overall; derivatives have different on-chip capacities. XDATA is an address space, not proof that a physical RAM chip exists.

Internal RAM region	Role and example
00H–1FH	Four banks: bank 0 = 00H–07H; bank 1 = 08H–0FH; bank 2 = 10H–17H; bank 3 = 18H–1FH.
20H–2FH	Sixteen byte locations, also 128 separately addressable bits (bit addresses 00H–7FH).
30H–7FH	General scratch and stack storage; allocate without collisions.

RS1:RS0 = 00, 01, 10, 11 selects banks 0, 1, 2, 3. Direct address 05H always denotes RAM 05H; `R5` denotes the selected bank's R5. In bank 1 it is RAM 0DH. All guide ALPs assume bank 0; those that change PSW explicitly establish it.

Direct addresses 80H–FFH select SFRs. Original 8051 has no upper 128-byte RAM; an 8052 provides it through indirect access. `MOV A,90H` reads P1; `MOV R0,#90H; MOV A,@R0` does *not* read P1 and has no valid upper-RAM target on the baseline.

2.3 Byte address versus bit address

RAM byte 20H contains bit addresses 00H–07H; RAM byte 21H contains 08H–0FH. Formula: byte = 20H + floor(bit address/8), bit position = bit address modulo 8. Thus `MOV 20H,#01H` changes a byte, while `SETB 20H` sets bit address 20H, which is RAM byte 24H, bit 0.

Implemented SFRs whose byte addresses end in 0H or 8H may be bit-addressable. P1 byte address is 90H; P1.2 bit address is 92H. TMOD at 89H is *not* bit-addressable. Do not invent `SETB TMOD.0`.

Exam angle. Draw the RAM map, identify a bank's Rn address, or distinguish `MOV A,30H` from `MOV A,#30H`.

Common mistake. An address is not its contents; an identical address in two spaces is not the same storage cell. *Source check: Intel MCS-51 User's Manual, Architectural Overview / Memory Organization and Programmer's Guide; Module 1 deck used for prerequisite context..*

3 Assembly source, data representation and directives

3.1 An assembly statement

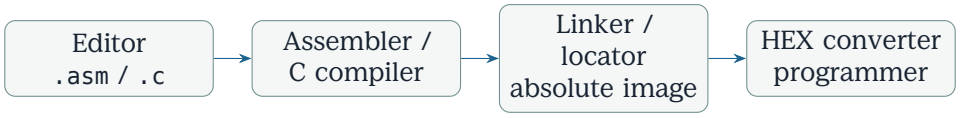
A statement can contain a label, mnemonic or directive, operands and a comment. For instructions with two ordinary operands the usual source order is `destination,source`. A label names the current code address; it is not executed. Mnemonics are symbolic names; an *opcode* is the encoded machine byte.

```
NEXT: MOV A,#24H      ; load literal 24H into A
      MOV A,24H       ; read the byte at internal RAM 24H

a_const EQU 24H       ; assembler symbol, not a CPU instruction
```

These are instructional statements, not a complete application. `#` marks a literal, `@` marks an indirect/indexed operand, a comma separates operands, and `;` starts an A51 comment. `MUL AB` and `DIV AB` use the special fixed operand pair; `MUL A,B` is not the instruction syntax. A51 is not case-sensitive for these identifiers, but use consistent capitals.

3.2 From source to execution



A listing shows source, addresses and generated bytes; an object file also carries symbols and relocation information. The linker resolves references and assigns final addresses. Intel HEX is a text record format carrying bytes and addresses for loading program memory; its file size is not the same as occupied program-ROM size. Flash programming and simulation are separate checks. An `.exe` in a generic slide flowchart is not the normal 8051 program image.

Exam angle. Explain the workflow and identify which phase detects an invalid mnemonic versus an unresolved external symbol. *Source check: Class 1 pp. 5–24, Class 2 p. 2; Keil A51 documentation / assembler statements and linker documentation..*

3.3 Numbers, bytes, signed values, BCD and ASCII

An unsigned byte represents 0–255 (00H–FFH). Hexadecimal needs two digits per byte; each digit represents four bits. In A51, `25H` = decimal 37; `25` = decimal 25; `1010B` = decimal 10. Prefix a leading zero when a hex literal begins with A–F: `0FFH`, `0B83EH`. In C use `0xFF`; `55H` is not C syntax. C leading-zero literals are octal: `010` is decimal 8, not decimal 10.

Two's complement interprets an n-bit pattern x as x for x below 2^{n-1}, otherwise as $x - 2^n$. For 8 bits, FFH = $255 - 256 = -1$ and 80H = $128 - 256 = -128$. For 16 bits, FF9CH = $65436 - 65536 = -100$. The negative of a nonzero n-bit value v is $(2^n - v)$ mod 2^n. A sign bit plus "remaining magnitude bits" is *not* a correct general description of two's complement: FFH does not mean -127.

Packed BCD stores one decimal digit per nibble: decimal 58 is 58H in BCD, but decimal 58 is 3AH in ordinary binary. A valid BCD nibble is 0–9. ASCII digit '5' is 35H, not 05H; 'A' is 41H. `DB 'HELLO'` emits five character bytes in CODE; it does not automatically append a zero terminator.

Representation	Example and interpretation
Unsigned byte	E7H = 231 decimal.
Signed byte	E7H = $231 - 256 = -25$.
Packed BCD byte	97H = decimal 97; E7H is invalid packed BCD.
ASCII bytes	'97' = 39H, 37H; two characters, not one numeric byte.
16-bit stored quantity	1234H can be stored low first as RAM[30H]=34H and RAM[31H]=12H, if the program adopts that convention.

Common mistake. The processor stores a bit pattern. Its intended signed, unsigned, BCD or character interpretation comes from the program.

3.4 Assembler directives

Directives control assembly rather than execute on the CPU. `DB` emits bytes, but no opcode representing "execute DB"; `EQU` emits no data by itself.

Directive	Example	Effect
ORG	`ORG 0100H`	Sets the location counter within the current segment; use absolute CODE here.
EQU	`COUNT EQU 05H`	Defines a constant symbol; write `MOV R7,#COUNT` to load it. No # in the EQU expression.
DB	`TABLE: DB 12H,34H`	Emits bytes in CODE; table label points to its first byte.
END	`END`	Ends assembly of the source module; all routines and data must appear before it.
DATA / BIT	`TEMP DATA 30H`	Names a byte address; `FLAG BIT 00H` names a bit address. Neither initializes hardware.

`ORG` is not a runtime jump. If a program begins at 4100H it needs a monitor that starts execution there or a reset-vector jump to that location. Examples here start at 0000H. A CPU stop is normally an infinite loop; `END` does not halt execution. Protect subroutines and constant tables from accidental fall-through. *Source check: Class 1 pp. 20–24; Keil DB statement..*

3.5 Instruction bytes are not instruction cycles

The complete operand form determines length and timing. The simplified slide rules “no number means one byte” and “an 8-bit number means two bytes” fail, for example, for `MOV 30H,#55H` (3 bytes).

Instruction	Machine bytes	Bytes	Cycles	Interpretation
`MOV A,R5`	ED	1	1	Selected-bank R5.
`MOV A,#25H`	74 25	2	1	Literal.
`MOV A,25H`	E5 25	2	1	Direct RAM.
`MOV 30H,#55H`	75 30 55	3	2	Direct destination plus immediate.
`MOV 30H,31H`	85 31 30	3	2	Encoded source precedes destination.
`MOV DPTR,#1234H`	90 12 34	3	2	High operand byte precedes low.
`LJMP 1234H`	02 12 34	3	2	High address byte precedes low.
`MOVC A,@A+DPTR`	93	1	2	CODE lookup.
`MOVX @DPTR,A`	F0	1	2	XDATA write.
`MUL AB / DIV AB`	A4 / 84	1	4	Byte arithmetic using A and B.

One classic machine cycle is twelve oscillator periods. A 1-byte instruction can take four cycles; a 3-byte instruction can take two. Instruction operand byte order, a program’s RAM storage convention and stack return-address order are distinct matters. *Source check: Keil opcode/length table; Intel instruction definitions; TEMIC C51 instruction definitions..*

4 Addressing modes and operand reasoning

Addressing describes how the instruction finds an operand. Identify each operand separately: `MOV 30H,#55H` has a direct destination and an immediate source.

Mode	Representative statement	Meaning
Immediate	`MOV A,#25H`	Literal 25H is part of instruction encoding.
Register	`MOV A,R5`	Read current bank's R5.
Direct	`MOV A,25H`	Operand byte contains internal RAM/SFR address.
Register indirect	`MOV A,@R0`	R0 contains an internal RAM address; R0/R1 only.
Indexed CODE	`MOVC A,@A+DPTR`	Read CODE[DPTR + unsigned A]. A becomes table value.
Indexed CODE, PC	`MOVC A,@A+PC`	Uses PC after the 1-byte MOVC has advanced it.
Bit direct	`SETB P1.2`	Addresses one implemented bit; distinct from byte 92H.
Relative branch	`SJMP LOOP`	Signed displacement added to following instruction's PC.

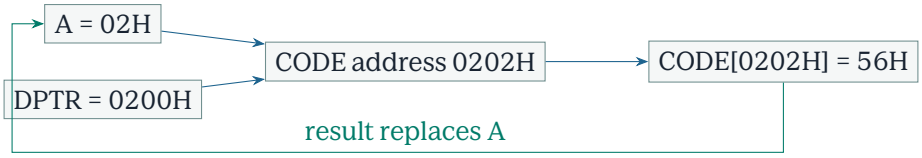
The first five are the classroom data-addressing modes. Bit and branch addressing explain supporting instructions. `MOVX A,@DPTR` accesses XDATA using a 16-bit pointer; `MOVX A,@R0` uses an 8-bit pointer and external-bank hardware convention (typically P2 latch as the high address). Neither is ordinary internal-RAM `MOV`.

4.1 A worked value/address trace

Assume RAM[25H]=A7H, R0=25H and R5=19H, bank 0. Independently executing the following statements gives:

Statement	A after execution
`MOV A,#25H`	25H; reads no RAM operand.
`MOV A,25H`	A7H; reads RAM byte 25H.
`MOV A,@R0`	A7H; pointer value is 25H.
`MOV A,R5`	19H; selected register contents.

For an indexed table at CODE 0200H containing 12H, 34H, 56H, 78H, DPTR=0200H and A=02H produces effective address 0202H and A=56H. DPTR remains 0200H. The original index in A is overwritten.



Common mistake. `@R2` is not valid for byte indirect MOV. `MOV R2,R3` is not a native register-to-register operand form; use A as an intermediate, or explicitly use bank RAM addresses. In slide tables, `Rx/Ry` ambiguously means a direct address; this guide writes `direct` or a concrete address.

Exam angle. Identify modes and resulting values; explain why `MOVC` cannot write a table and why `MOVX` requires a different memory space. *Source check: Class 1 pp. 35–46; Intel / Keil instruction addressing conventions and MOVC definition..*

5 Complete data-transfer programs

For every ALP, the algorithm is expressed in the explanation and the selectable source; the terminal `SJMP` preserves results for inspection. Standard register symbols come from `$MOD51`. All hexadecimal addresses are explicit. External-memory tests assume a simulator XDATA model or correctly connected RAM.

5.1 A01: Immediate, register, direct and indirect transfers

Problem and assumptions. Transfer 55H among A, R3 and internal RAM 30H–32H in bank 0. These scratch locations do not overlap registers or the stack.

Algorithm. Load a literal; copy to R3 and RAM; load R0 with the RAM address; read indirectly; then copy by the two direct destination forms.

```
$MOD51
ORG 0000H
MOV PSW,#00H          ; select bank 0
MOV A,#55H            ; immediate value
MOV R3,A              ; register transfer
MOV 30H,A             ; direct RAM write
MOV R0,#30H           ; R0 holds an address
MOV A,@R0             ; indirect RAM read
MOV 31H,R3            ; direct destination
MOV 32H,30H           ; direct-to-direct copy
STOP: SJMP STOP
END
```

Expected result. A=55H, R3=55H, R0=30H and RAM[30H]=RAM[31H]=RAM[32H]=55H. Ordinary MOV leaves CY, AC and OV unchanged; parity follows A. The last MOV has encoded bytes 85H,30H,32H (source first in the encoding).

5.2 A02: CODE table lookup using MOVC

Problem. Select the third entry of a four-byte constant table and store it in RAM 30H. **Algorithm.** Point DPTR to the table, load zero-based index 2, fetch CODE and save A. The stop loop protects the table from execution.

```
$MOD51
ORG 0000H
MOV DPTR,#TABLE
MOV A,#02H            ; zero-based table index
MOVC A,@A+DPTR        ; read CODE, not DATA
MOV 30H,A
STOP: SJMP STOP        ; prevent execution of DB bytes
TABLE: DB 12H,34H,56H,78H
END
```

Expected result. A and RAM[30H]=56H. DPTR remains the table base. This is indexed CODE access, even if the constant table resides on the same chip as the instructions. `DB` did not initialize internal RAM.

5.3 A03: XDATA write and readback using MOVX

Problem. Write 5AH to XDATA 4500H, read it back and store the result at internal RAM 30H. **Algorithm.** Establish the address in DPTR; write A with MOVX; clear A; read with MOVX; save to internal RAM.

```
$MOD51
ORG 0000H
MOV DPTR,#4500H       ; XDATA address
MOV A,#5AH
MOVX @DPTR,A          ; write external data
CLR A
MOVX A,@DPTR          ; read it back
MOV 30H,A
STOP: SJMP STOP
END
```

Expected result. XDATA[4500H]=5AH and RAM[30H]=5AH. On an original external bus, P0 multiplexes low address/data, P2 carries the high address and P3.6/P3.7 supply write/read strobes. P0/P2 cannot simultaneously serve as independent pattern outputs in this application. A modern derivative's internal XRAM is device-specific.

5.4 A04: Copy 17 bytes to external memory

Problem. Copy inclusive internal range 20H–30H to XDATA 1020H–1030H. Length is $30\text{H} - 20\text{H} + 1 = 11\text{H} = 17$. The source and destination are different spaces; there is no physical overlap in the stated hardware.

Algorithm. First initialize a reproducible source 00H–10H. Reset R0 and DPTR to the two starting addresses and transfer exactly 17 bytes, incrementing both pointers after each transfer. R7 is a remaining-count register.

```
$MOD51
ORG 0000H
MOV PSW,#00H
MOV R0,#20H
MOV R7,#11H           ; 17 decimal bytes
CLR A
INIT: MOV @R0,A       ; source = 00H ... 10H
INC A
INC R0
DJNZ R7,INIT
MOV R0,#20H
MOV DPTR,#1020H
MOV R7,#11H
COPY: MOV A,@R0
MOVX @DPTR,A
INC R0
INC DPTR
DJNZ R7,COPY
STOP: SJMP STOP
END
```

Expected result. XDATA[1020H+i]=i for $i = 0, \ldots, 16$; final R0=31H, DPTR=1031H, R7=00H and A=10H. The original slide's `#0AH` transfers only 20H–29H to 1020H–1029H. Either shorten its stated ranges or use the corrected count here.

For copying within the *same* RAM, overlapping blocks require direction-aware copying: if the destination begins inside and above the source, copy backward to avoid overwriting unread bytes. The

simple forward loop is safe for nonoverlapping blocks, or appropriately ordered overlap. A zero initial DJNZ byte count causes 256 iterations, not zero; guard a variable-length request before entering the loop.

5.5 A05: Exchange two nonoverlapping five-byte blocks

Problem. Exchange RAM 30H–34H with 40H–44H. The program initializes the first block to 11H–15H and the second to A1H–A5H. Length=05H; both inclusive ranges remain within classic RAM and do not overlap.

Algorithm. For each pair, save the first byte in R4, place the second in the first location, then restore the saved first byte into the second. Advance R0/R1 and decrement R7.

```
$MOD51
ORG 0000H
MOV PSW,#00H
MOV R0,#30H
MOV R1,#40H
MOV R7,#05H
MOV A,#11H
INIT: MOV @R0,A
ADD A,#90H
MOV @R1,A
SUBB A,#8FH          ; C=0 here: next low-block value
INC R0
INC R1
DJNZ R7,INIT
MOV R0,#30H
MOV R1,#40H
MOV R7,#05H
SWAPBLOCK: MOV A,@R0
MOV R4,A             ; save first byte
MOV A,@R1
MOV @R0,A
MOV A,R4
MOV @R1,A
INC R0
INC R1
DJNZ R7,SWAPBLOCK
STOP: SJMP STOP
END
```

Expected result. RAM[30H–34H]=A1H, A2H, A3H, A4H, A5H; RAM[40H–44H]=11H, 12H, 13H, 14H, 15H. Final pointers 35H and 45H, R7=0, R4=15H. A scratch register prevents loss of the first byte. For pair 0: R4=11H, first location becomes A1H, second becomes 11H. *Source check: Class 2 pp. 8–12 recovered from images; corrected A51 notation and address/count assumptions. Intel MOV/MOVX instruction definitions..*

6 Arithmetic, PSW flags and execution traces

Arithmetic works on bit patterns; flags tell you about carry, borrow and signed range. Preserve a result before a later instruction overwrites A or B. Saving A using MOV does not destroy CY. In multi-byte operations, process low bytes first and propagate CY into higher bytes.

6.1 PSW and flag rules

7	6	5	4	3	2	1	0
CY	AC	F0	RS1	RS0	OV	–	P

PSW

Flag	Meaning
CY	ADD/ADDC: carry out of bit 7. SUBB: unsigned borrow out. Also the single-bit accumulator for bit operations.
AC	Carry from bit 3 to bit 4 in addition; borrow across the low-nibble boundary in subtraction. Used by decimal adjustment.
OV	Signed arithmetic result cannot fit in -128–127; carry/borrow into bit 7 differs from carry/borrow out.
P	Hardware-maintained odd parity of A: 1 for an odd number of 1 bits, 0 for even. It is not parity of B or the whole RAM.

F0 is a software flag; RS1/RS0 select a bank. PSW.1 is unused/reserved in the baseline diagram; a derivative may define it differently. Do not use it as a portable status flag.

Instruction	Result and important flags
ADD A,src	$A \leftarrow (A + src) \bmod 256$; incoming CY ignored. CY, AC, OV updated.
ADDC A,src	$A \leftarrow (A + src + CY_{in}) \bmod 256$; new CY propagates to the next byte.
SUBB A,src	$A \leftarrow (A - src - CY_{in}) \bmod 256$; incoming CY is borrow. Clear CY for a fresh subtraction.
INC / DEC	Wrap modulo 256 for bytes; CY/AC/OV are unchanged. INC DPTR wraps modulo 65536; no DEC DPTR instruction.
MUL AB	Unsigned product: B=high byte, A=low byte. CY=0; OV=1 if high byte is nonzero, otherwise 0. AC undefined.
DIV AB	If B is nonzero: A=quotient, B=remainder; CY=0, OV=0. If B=0: CY=0, OV=1, A/B undefined. AC undefined.
DA A	Packed-BCD adjustment after ADD/ADDC; not general binary-to-BCD conversion or a subtraction correction. CY reports decimal carry; AC/OV are not new decimal-status flags.

Parity updates when A changes, including after MOV, INC, MUL and DIV. Undefined flags are not safe branch conditions.

6.2 How to calculate flags

For ADD/ADDC, calculate the full unsigned sum and its low nibble separately. CY=1 if full sum exceeds FFH; AC=1 if nibble sum exceeds FH. Calculate signed operands separately for OV. For SUBB, include incoming borrow when checking both full and nibble subtraction.

Operation	A	CY	AC	OV	P	Interpretation
7FH + 01H	80H	0	1	1	1	$127 + 1$ exceeds signed range.
FFH + 01H	00H	1	1	0	0	Unsigned carry; $-1 + 1 = 0$.
80H − 01H, CYin=0	7FH	0	1	1	1	Signed $-128 - 1$ overflows.
00H − 01H, CYin=0	FFH	1	1	0	0	Unsigned borrow; signed result -1.
25H − 34H, CYin=0	F1H	1	0	0	1	$37 - 52 = -15$.

Exam angle. A flag question usually asks for A, CY, AC, OV and P. Give all requested flags and explain signed overflow separately from unsigned carry. *Source check: Intel arithmetic instruction definitions; Keil ADDC; manufacturer C51 instruction definitions..*

6.3 A06: Classroom addition of 25H, 34H and 12H

Problem. Add the exact classroom literals and trace machine bytes and flags. Bank 0 is the reset bank; there are no prior bank changes. **Algorithm.** Load R5/R7, clear A through MOV, add each of the three operands, then hold the result.

```
$MOD51
ORG 0000H
MOV R5,#25H
MOV R7,#34H
MOV A,#00H
ADD A,R5
ADD A,R7
ADD A,#12H
HERE: SJMP HERE
END
```

Address	Bytes	Instruction	Result
0000H	7D 25	MOV R5,#25H	R5=25H
0002H	7F 34	MOV R7,#34H	R7=34H
0004H	74 00	MOV A,#00H	A=00H, P=0
0006H	2D	ADD A,R5	A=25H; C/AC/OV=0, P=1
0007H	2F	ADD A,R7	A=59H; C/AC/OV=0, P=0
0008H	24 12	ADD A,#12H	A=6BH; C/AC/OV=0, P=1
000AH	80 FE	SJMP HERE	PC stays at 000AH

Expected result. 6BH = decimal 107. Code at 0000H–000BH is exactly 12 bytes, not 12 cycles. The last branch uses offset FEH = -2 from next PC=000CH. `END` emits no instruction byte.

6.4 A07: An 8-bit addition with a preserved carry byte

Problem. Add F0H and 30H and store a 16-bit result at XDATA 4500H–4501H, low byte first. **Algorithm.** ADD gives A=20H and CY=1; store the low byte; move zero to A without changing CY; ADDC zero converts incoming carry to the high byte.

```
$MOD51
ORG 0000H
MOV A,#0F0H
ADD A,#30H           ; result 0120H
MOV DPTR,#4500H
MOVX @DPTR,A        ; low byte 20H; C unchanged
INC DPTR
MOV A,#00H          ; MOV does not change C
ADDC A,#00H         ; turn carry into high byte
MOVX @DPTR,A        ; high byte 01H
STOP: SJMP STOP
END
```

Expected result. XDATA[4500H]=20H, XDATA[4501H]=01H, representing 0120H=288. ADD flags are CY=1, AC=0, OV=0, P=1. After `ADDC A,#00H`, A=01H and the *new* CY=0; the old carry has already been saved into the high result. `CLR C` before ADD would be harmless but unnecessary; ADD ignores incoming CY.

6.5 A08: Multi-byte addition with propagated carry

Problem. Add 1234H and 00F0H; store low/high at RAM 30H/31H and a beyond-16-bit carry at 32H. **Algorithm.** ADD low bytes, preserve result, then ADDC high bytes; collect the final carry separately.

```
$MOD51
ORG 0000H
MOV A,#34H           ; low byte of 1234H
ADD A,#0F0H          ; low byte of 00F0H
MOV 30H,A            ; 24H; C=1
MOV A,#12H
ADDC A,#00H          ; include low-byte carry
MOV 31H,A            ; 13H
MOV A,#00H
ADDC A,#00H
MOV 32H,A            ; carry beyond 16 bits: 00H
STOP: SJMP STOP
END
```

Step	Result
34H + F0H	124H: low 24H, CY=1, AC=0, OV=0.
12H + 00H + 1	13H: high 13H, CY=0, AC=0, OV=0.
Save final carry	00H; complete result 1324H.

Expected result. RAM[30H]=24H, [31H]=13H, [32H]=00H. MOV between low and high steps preserves CY. Using ADD on the high byte would incorrectly produce 1224H.

6.6 A09: Subtraction with an explicit initial borrow

Problem. Subtract 34H from 25H and store result plus borrow. **Algorithm.** CLR C removes any prior borrow; SUBB subtracts the operand and current CY; save A and CY. Bit address 00H is RAM 20H.0, outside the result byte.

```
$MOD51
ORG 0000H
CLR C                ; no incoming borrow
MOV A,#25H
SUBB A,#34H          ; 25H - 34H = F1H, borrow=1
MOV 30H,A
MOV 00H,C            ; bit 00H = RAM 20H.0
STOP: SJMP STOP
END
```

Expected result. RAM[30H]=F1H, bit 00H=1. CY=1, AC=0, OV=0, P=1. Interpreted signed, F1H is -15; interpreted unsigned it is 241. If CY were 1 on entry, the operation would be $25H - 34H - 1 = F0H$. SUBB does not always subtract an extra 1.

6.7 A10: A 16-bit subtraction

Problem. Compute 1200H − 00F1H, low result at RAM 30H, high at 31H, final unsigned borrow at bit 00H. **Algorithm.** Clear CY once before the least significant byte; propagate each subsequent SUBB borrow.

```
$MOD51
ORG 0000H
CLR C
MOV A,#00H           ; low byte of 1200H
SUBB A,#0F1H         ; low byte of 00F1H
MOV 30H,A            ; 0FH; borrow=1
MOV A,#12H
SUBB A,#00H          ; subtract propagated borrow
MOV 31H,A            ; 11H; final borrow=0
MOV 00H,C
STOP: SJMP STOP
END
```

Expected result. Low: $00H - F1H = 0FH$ with CY=1 and AC=1. High: $12H - 00H - 1 = 11H$ with final CY=0, AC=0, OV=0. Result=110FH; RAM[30H]=0FH, [31H]=11H, bit 00H=0. Clearing C between bytes would incorrectly produce 120FH.

6.8 A11: Multiplication with both result bytes preserved

Problem. Multiply 0AH by 88H and store product low/high in XDATA 4500H/4501H. **Algorithm.** Load A/B; MUL AB; store low A; move high B into A; store at the next address.

```
$MOD51
ORG 0000H
MOV A,#0AH
MOV B,#88H
MUL AB              ; B:A = 0550H
MOV DPTR,#4500H
MOVX @DPTR,A        ; low byte first by convention
INC DPTR
MOV A,B             ; preserve original high result
MOVX @DPTR,A
STOP: SJMP STOP
END
```

Expected result. $10 \times 136 = 1360 = 0550H$, so low=50H, high=05H. Immediately after MUL: A=50H, B=05H, CY=0, OV=1, P=0; AC undefined. MOV A,B later makes A=05H, still even parity. The slide's "move 88 to A" comment is wrong: the destination is B. The high result must not be discarded.

6.9 A12: Division, quotient/remainder and zero divisor

Problem. Divide 65H by 08H, storing quotient/remainder in XDATA 4500H/4501H. RAM 32H is a software error status. **Algorithm.** Save the dividend, test B using A/JZ, restore dividend only for nonzero B, divide, then store both results. For B=0, do not execute DIV; set status=1 and define zero/zero output as an error sentinel.

```
$MOD51
ORG 0000H
MOV 32H,#00H        ; software status: 0 = OK
MOV A,#65H
MOV B,#08H          ; change to 00H to test error
MOV R7,A            ; save dividend for zero check
MOV A,B
JZ ZERO_DIVISOR
MOV A,R7
DIV AB              ; A=quotient, B=remainder
SJMP STORE
ZERO_DIVISOR:
MOV 32H,#01H        ; define a software error result
CLR A
MOV B,#00H
STORE: MOV DPTR,#4500H
MOVX @DPTR,A
INC DPTR
MOV A,B
MOVX @DPTR,A
STOP: SJMP STOP
END
```

Expected result. $101/8 = 12$ remainder 5: quotient 0CH, remainder 05H, status 00H. Immediately after successful DIV: CY=0, OV=0, A=0CH, B=05H, P=0, AC undefined. With the source changed to `MOV B,#00H`, outputs are 00H/00H and status=01H. This defined software behavior must not be confused with hardware DIV-by-zero's undefined A/B.

Class 2 p. 10's input table repeats multiplication operands 0A/88 while its instructions and quotient/remainder match 65H/08H. Without the H suffix, #65 means decimal 65 and would give quo-

tient 08H, remainder 01H when divided by decimal 8. The output evidence supports the labelled hex reconstruction used here.

6.10 A13: Decimal adjustment and increment/decrement

Problem. Add packed-BCD 58 and 67, preserve decimal carry, then demonstrate byte wrapping without changing CY. **Algorithm.** ADD, immediately DA A, save result and CY; load FFH, increment to 00H, then decrement to FFH.

```
$MOD51
ORG 0000H
MOV A,#58H           ; packed BCD 58
ADD A,#67H           ; packed BCD 67
DA A                 ; decimal 125 -> C=1, A=25H
MOV 30H,A
MOV 00H,C            ; save decimal hundreds carry
MOV A,#0FFH
INC A                ; wraps to 00H; C still 1
MOV 31H,A
DEC A                ; wraps to FFH; C still 1
MOV 32H,A
STOP: SJMP STOP
END
```

Expected result. Binary ADD produces BFH; the low nibble F exceeds 9, so adjustment adds 06H, giving C5H. The high decimal digit requires adding 60H; A becomes 25H with CY=1, representing decimal 125 as carry 1 and BCD 25. RAM[30H]=25H, bit 00H=1, [31H]=00H, [32H]=FFH. INC/DEC leave CY=1. Their parity changes with A; FFH has P=0.

In general DA A adjusts the low nibble when it exceeds 9 or AC is set, and the high decimal digit when needed or CY is set; decimal carry is preserved/established. It is meaningful after addition of valid packed BCD, including ADDC in multi-byte BCD. `DA A` after arbitrary binary input or SUBB does not perform a general conversion. For example 45H+38H gives 7DH before adjustment and 83H after adjustment (decimal 83). *Source check: Class 1 pp. 63–72; Class 2 pp. 3–10; lab manual arithmetic exercises. Intel / manufacturer ADD, ADDC, SUBB, MUL, DIV and DA definitions..*

7 Supporting flow control, calls, stack and software delay

Loops make block transfer possible; conditions manage carry/borrow and inputs; a call reuses a routine. These are supporting skills for the specified programs.

7.1 The jumps you need

Form	Bytes	Cycles	Meaning and limit
SJMP label	2	2	Unconditional; offset -128 to $+127$ from next instruction.
LJMP address	3	2	Any 16-bit CODE address.
AJMP address	2	2	Same 2 KB page as *following* instruction.
JZ / JNZ label	2	2	Test A=0 / A$\neq$0; no general zero flag.
JC / JNC label	2	2	Test CY=1 / CY=0.
JB / JNB bit,label	3	2	Test addressed bit=1 / 0.
JBC bit,label	3	2	If bit=1, clear it and jump. Port operation is read-modify-write.
DJNZ Rn,label	2	2	Decrement Rn; branch if its new value is nonzero.
DJNZ direct,label	3	2	Same operation on directly addressed byte.
CJNE A,#data,label	3	2	Jump if unequal; CY=1 if unsigned A$<$data, otherwise 0.

All conditional branch displacements are relative to their following instruction. DJNZ tests *its operand*, not A, and does not set arithmetic flags. CJNE also has A,direct; Rn,#data; @Ri,#data forms. It is an unsigned comparison and changes CY; do not insert it into a carry-propagation chain without preserving CY. RETI belongs to interrupt service, not an ordinary subroutine.

7.2 Relative offsets and 2 KB boundaries

A SJMP at 0100H targeting 00F0H has next PC=0102H, offset=$00F0H - 0102H = -18 = EEH$. Encode 80H,EEH. Conversely offset 7FH at 0100H targets 0181H. Offset 80H means -128, not $+128$.

For AJMP/ACALL at address p, the legal target satisfies

$$(\text{target} \,\&\, \text{F800H}) = ((p+2) \,\&\, \text{F800H}).$$

At 07FEH the following instruction begins at 0800H, so the target page is 0800H–0FFFH. Target 0123H is illegal there even though it lies in the page containing the jump's own opcode. LCALL/LJMP remove this page restriction. AJMP is 2 bytes; an absolute 11-bit address is packed across the op-code and the second byte. *Source check: Class 3 pp. 2–11; Keil AJMP page rule..*

7.3 Stack operation

SP points to the last used internal-RAM stack byte. PUSH increments SP, then writes; POP reads at SP, then decrements it. Reset SP=07H would place the first pushed byte at 08H, overlapping bank 1; relocate it if that bank or nearby storage is used. A classic stack has no automatic bounds protection.

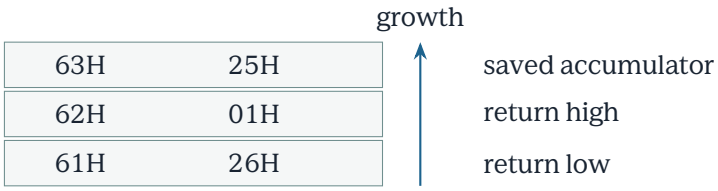
CALL pushes the following instruction's return address **low byte first, then high**; RET pops high then low and restores the PC. LCALL is 3 bytes/2 cycles; ACALL is 2 bytes/2 cycles; RET is 1 byte/2 cycles. A call does not automatically save A, B, DPTR, PSW or Rn; the caller/callee must agree about preservation.

7.4 A14: A call with an exact stack trace

Problem. Call a routine at 0345H from LCALL at 0123H, preserve A=25H and copy it to R1 after return. Stack is 61H upward, outside this program's scratch area. **Algorithm.** Reset jumps to initialization; call saves 0126H; subroutine saves ACC, temporarily changes A, restores it and returns.

```
$MOD51
ORG 0000H
LJMP START
ORG 0100H
START: MOV SP,#60H   ; stack begins at RAM 61H
MOV A,#25H
LJMP CALLSITE
ORG 0123H
CALLSITE: LCALL SUBR ; return address = 0126H
MOV R1,A
STOP: SJMP STOP
ORG 0345H
SUBR: PUSH ACC       ; preserve A
MOV A,#55H
POP ACC
RET
END
```

Event	SP	Stack / CPU state
Before LCALL at 0123H	60H	A=25H; next address=0126H.
LCALL pushes low then high	62H	RAM[61H]=26H, RAM[62H]=01H; PC=0345H.
PUSH ACC	63H	RAM[63H]=25H.
MOV A,#55H	63H	A=55H; return bytes intact.
POP ACC	62H	A=25H.
RET	60H	PC=0126H, then MOV R1,A gives R1=25H.



Expected result. SP=60H, A=25H, R1=25H. Popped RAM bytes may remain physically present; popping changes SP, not an erase operation. Source `END` appears *after* SUBR. An END before SUBR would exclude the routine from assembly. Falling into a subroutine without a matching call would leave RET without a valid return address.

7.5 Counted software delay and a valid whole-port toggle

A software delay spends CPU instruction cycles. It blocks other foreground work, and its time depends on the exact instruction sequence, processor clock and interrupts.

7.6 A15: Port-latch toggle with a 100-count delay

Problem. Alternate all P1 bits using a counted delay on a classic core at 11.0592 MHz. P1 outputs drive logic or resistor-limited LEDs within device current limits. **Algorithm.** Initialize P1, XOR its latch with FFH, call DELAY and jump back; DELAY loads R7 with decimal 100 and executes DJNZ exactly 100 times.

```
$MOD51
ORG 0000H
MOV SP,#60H
MOV P1,#00H
BACK: XRL P1,#0FFH  ; complement entire port latch
LCALL DELAY
SJMP BACK
DELAY: MOV R7,#100   ; decimal 100
L1: DJNZ R7,L1
RET
END
```

Delay instruction	Cycles each	Executions	Total
MOV R7,#100	1	1	1
DJNZ R7,L1	2	100	200
RET	2	1	2
Routine entry through RET			203
LCALL to routine	2	1	2
Call plus routine			205

At 11.0592 MHz, $t_{MC} = 12/11{,}059{,}200$ s exactly. Routine time=$203t_{MC} = 220.269097$ μs; including LCALL=$205t_{MC} = 222.439236$ μs. At 12 MHz these are 203 and 205 μs. Class 4 p. 10's 220.255 μs uses an early 1.085 μs approximation and excludes the call.

For this particular complete loop, edge-to-edge instruction cost is LCALL (2) + routine (203) + SJMP (2) + next XRL direct,#data (2) = **209 cycles**. Thus half-period is 226.779514 μs and ideal software-loop square-wave frequency is approximately 2204.785 Hz at 11.0592 MHz. This analysis is valid because the delay is entirely software and no interrupts intervene.

`CPL P1` is not a whole-byte instruction. Since P1's numeric byte address is also the bit address of P1.0, permissive assembly could complement only that bit; it cannot mean all eight bits. `XRL P1,#0FFH` performs the intended read-modify-write on the latch. DS89C420 instruction timings differ from the classic cycle table; the slide's DS89C420 heading cannot justify a classic loop delay.

7.7 A16: Classroom 55H/AAH output on P0

Problem. Reconstruct Class 4 p. 7 as a complete application including its missing delay routine. Use P0 with external pull-ups for high levels; no external memory bus. **Algorithm.** Write 55H, delay, write AAH, delay, repeat.

```
$MOD51
ORG 0000H
MOV SP,#60H
MOV P0,#0FFH          ; external pull-ups required
BACK: MOV P0,#55H
LCALL DELAY
MOV P0,#0AAH
LCALL DELAY
SJMP BACK
DELAY: MOV R7,#100
L1: DJNZ R7,L1
RET
END
```

Expected result. P0 alternates 01010101B and 10101010B. Call-plus-delay is 205 cycles at the stated classic clock, but the loop jump occurs on only one of the two edge paths; the complete output intervals are not exactly equal. This is a pattern example, not a calibrated waveform generator. Use a timer for a prescribed interval.

7.8 A17: Monitor an input and generate a logic strobe

Problem. Wait until P1.2 is high, send 45H to P0 and send a high-to-low pulse on P2.3 (Class 4 p. 15). Assume a clean logic input, P0 pull-ups and no external bus. **Algorithm.** Release the input; poll it; write data; set/hold/clear the strobe; stop.

```
$MOD51
ORG 0000H
SETB P1.2             ; release switch input
CLR P2.3
WAIT_HIGH: JNB P1.2,WAIT_HIGH
MOV P0,#45H           ; external pull-ups for P0
SETB P2.3
NOP                   ; one-cycle pulse extension
CLR P2.3
STOP: SJMP STOP
END
```

Expected result. Once the input is sampled high, P0=45H and P2.3 produces one pulse, then remains low. The NOP provides one additional machine-cycle hold; an actual peripheral needs its own setup/hold/pulse-width specifications. A bouncing switch may need debouncing. The oven/buzzer slide (Class 4 p. 16) similarly teaches polling and a pulse; it lacks a defined audible frequency and load driver.

8 Embedded C: types, delay and I/O

C expresses algorithms more compactly than assembly, but the compiler still generates 8051 instructions. Hardware registers, memory spaces and clock timing remain important. The syntax here is **Keil C51**; an ordinary desktop C compiler does not understand its hardware extensions.

8.1 C51 data types and ranges

Type	Width	Range / purpose
`unsigned char`	8 bits	0–255; byte data, ports, small counters.
`signed char`	8 bits	−128–127; explicitly signed byte.
`unsigned int`	16 bits	0–65535; timer counts and longer loop bounds.
`signed int`	16 bits	−32768–32767; two's complement.
`unsigned long`	32 bits	0–4294967295; larger arithmetic.
`signed long`	32 bits	−2147483648–2147483647.
`float`, `double`	32 bits in C51	C51 double uses the same representation as float; do not assume desktop 64-bit double.
`bit`	One bit	C51 Boolean object in bit-addressable storage, values 0/1.

Use `unsigned char` rather than relying on plain `char` signedness, which depends on compiler settings. Integer widths are compiler-specific; this table describes C51, not every C implementation. Floating-point arithmetic is relatively costly on an 8-bit processor and is unnecessary for these integer timer routines.

8.2 Hardware and memory extensions

```
#include <reg51.h>           /* standard 8051 SFR declarations */
sbit SWITCH = P1^2;          /* declaration: bit address P1.2 */
sbit LED = P2^0;
bit ready;                  /* allocated Boolean object */
unsigned char code table[] = {0x12,0x34,0x56};
/* Illustrative declarations; not an additional full program. */
```

`sfr` declares an absolute 8-bit SFR, for example `sfr MYPORT = 0x90;`. `sbit` aliases a particular bit address: it does not allocate an 8-bit variable. `bit` allocates a Boolean object. `code` places constants in program memory; reading them can require MOVC. C51 also provides `data`, `idata`, `xdata`, `pdata` and `bdata` memory-space qualifiers. `bdata` objects can have named bit aliases; therefore "sbit can only ever alias an SFR" is too restrictive. SFR declarations name the SFR space, not general RAM 80H–FFH.

The caret in a `sbit` declaration is special C51 syntax. In an ordinary expression `x = P1 ^ 2;` it means **bytewise XOR with 02H**, not "read P1.2". Use the declared bit symbol when accessing that pin. Hardware names in `reg51.h` include A-independent port/timer SFRs and control bits; define your own T0/T1 pin aliases explicitly rather than assuming their names exist. *Source check: Class 5 pp. 2–12; Keil C51 type documentation and support article 663; Keil sbit documentation / support article 1175..*

8.3 Integer promotion, masks and Boolean inversion

A byte operand generally promotes to 16-bit `int` before arithmetic. For `unsigned char x = 0x55;`, `~x` can be a 16-bit result FF AAH. Assignment back to a byte retains AAH. Make width explicit: `x = (unsigned char)(x ^ 0xFFU);`. Inverting eight bits is different from logical inversion: `!0x55` is zero, while `~0x55` complements its bits.

For a Boolean pin, use `OUT = !IN;`: input zero gives output one, and input one gives output zero. C51 has special bit-expression behavior and can generate efficient complement instructions; this does not remove ordinary integer-promotion rules for byte expressions. Keil's complement note explains why the destination type and compiler-generated instructions matter.

Cast before shifting a high byte into a 16-bit result: `((unsigned int)hi << 8) | lo`. An explicit unsigned conversion avoids signed overflow/promotion surprises. Unsigned byte increment wraps 255 to 0; an 8-bit loop `for(i=0; i<300; ++i)` can never finish. A loop using `i <= 255` with an unsigned byte also never terminates.

Exam angle. Choose a type for 900 intervals, explain `sbit` versus `bit`, or distinguish XOR, complement and logical NOT.

Common mistake. A numeric bit address is not an ordinary C pointer. A51's `55H` literal is not a C literal. *Source check: Keil support article 2483 (integer promotion and complement); C51 type table..*

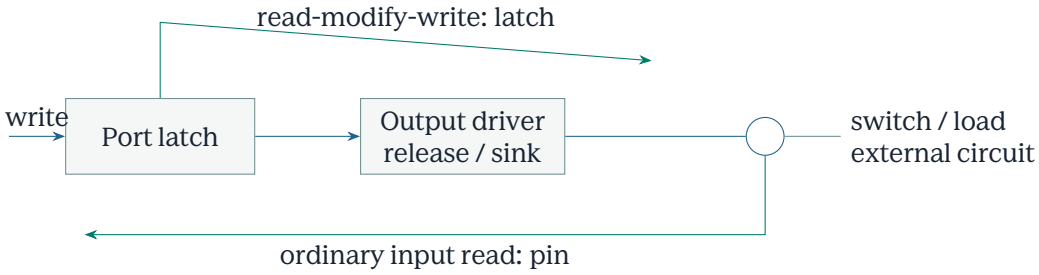
8.4 Time delay in C

An empty loop is not a time specification. Its generated instructions depend on compiler version, optimisation, memory model, variable width and clock; it can disappear entirely. `volatile` can force accesses, but still does not establish exact milliseconds. Hardware timing gives a reproducible count; the complete function still adds setup, polling, return and caller overhead.

C01–C03 and C05 include a complete Timer 0 helper. At **12 MHz on the classic core**, one timer tick is 1 μs. One nominal millisecond needs 1000 ticks; preload $65536 - 1000 = 64536 = FC18H$. The helper repeatedly stops, loads and polls Timer 0. Thus `delay_ms(250U)` means *250 hardware intervals of 1 ms plus software gaps*. It is adequate for a visibly slow classroom pattern, not an exact calibrated 250 ms edge interval. It owns Timer 0 and disables interrupt interference in examples that use it. Timer details and better continuous mode-2 accumulation follow in Section 10.

8.5 Ports: latch, pin and circuit

Ports P0–P3 are 8-bit SFRs at 80H, 90H, A0H and B0H. A latch stores the last written value. A pin is the external electrical node. They can differ, for example when an outside switch pulls a released pin low.



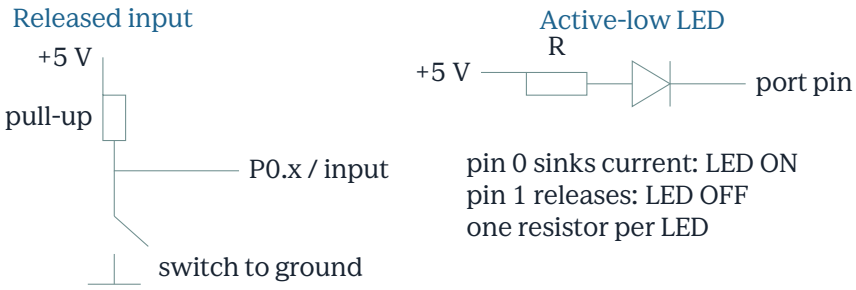
For classic P1/P2/P3 quasi-bidirectional inputs, write 1 to release the output driver before reading. Writing 0 actively sinks and prevents a switch's high level from being observed. Internal pull-ups support a released high level; external drive must obey device ratings. P3 alternate functions also require the corresponding latch bit to be 1.

P0 is open drain in ordinary GPIO operation. Writing 1 releases the pin; an external pull-up supplies a defined high level. Writing 0 sinks it. Its external-memory bus function is different; do not use bus pins simultaneously as independent GPIO in a MOVX circuit.

Read-modify-write instructions targeting a port read its *latch* so an external low does not accidentally rewrite unrelated outputs. Examples: `ANL P1,#mask`, `ORL P1,#mask`, `XRL P1,#0FFH`, `CPL P1.5`, `SETB P1.5`, `CLR P1.5`, `MOV P1.5,C`. An ordinary `MOV A,P1` or `MOV C,P1.2` reads the *pins*. A C expression such as `P1 = ~P1;` may compile as a pin read followed by a byte write; do not assume latch semantics without examining the generated assembly. A shadow byte, or an explicitly compiled

bit toggle, avoids unintended changes. *Source check: Intel User's Manual, I/O Ports / Read-Modify-Write Instructions; manufacturer programming manual, I/O port structure..*

Simple wiring and polarity



Illustrative LED sizing: for a 2.0 V LED, 0.2 V sink voltage and 2 mA target, $R = (5 - 2.0 - 0.2)/0.002 = 1400\,\Omega$; a 1.5 k$\Omega$ standard resistor gives about 1.87 mA. Confirm the selected part's per-pin and aggregate current ratings; use a buffer when needed. A buzzer or relay is driven through a suitable transistor/driver and supply; an inductive load needs a flyback path. An 8051 pin supplies a control signal, not arbitrary load current.

Common mistake. These are independent applications. Two functions that both configure Timer 0 cannot run concurrently without an ownership design.

8.6 C01: Alternate 55H and AAH on P1

Problem / assumptions. Alternate an 8-bit pattern at nominal 250 ms intervals; classic 12 MHz, Timer 0 reserved. Active-low LEDs display the inverse of the port bits. **Algorithm.** Write 55H, wait 250 one-millisecond hardware intervals, write AAH, repeat.

```
#include <reg51.h>
static void delay_ms(unsigned int ms)
{
    while (ms != 0U) {
        TR0 = 0;
        TMOD = (TMOD & 0xF0U) | 0x01U;
        TF0 = 0;
        TH0 = 0xFC; TL0 = 0x18;
        TR0 = 1;
        while (TF0 == 0) { }
        TR0 = 0;
        TF0 = 0;
        --ms;
    }
}
void main(void)
{
    EA = 0;
    while (1) {
        P1 = 0x55; delay_ms(250U);
        P1 = 0xAA; delay_ms(250U);
    }
}
```

Registers / result. TMOD low nibble=1; each load TH0=FCH, TL0=18H. P1 alternates 01010101 and 10101010. Full pattern cycle is approximately 500 ms (about 2 Hz), increased by software overhead.

8.7 C02: Toggle only P2.4

Problem. Toggle one output while preserving other P2 bits; classic 12 MHz. **Algorithm.** Declare the bit, initialize it low, toggle it and delay. External circuitry must not force the output pin to a conflicting level.

```
#include <reg51.h>
sbit OUT = P2^4;
static void delay_ms(unsigned int ms)
{
    while (ms != 0U) {
        TR0 = 0;
        TMOD = (TMOD & 0xF0U) | 0x01U;
        TF0 = 0;
        TH0 = 0xFC; TL0 = 0x18;
        TR0 = 1;
        while (TF0 == 0) { }
        TR0 = 0;
        TF0 = 0;
        --ms;
    }
}
void main(void)
{
    EA = 0;
    OUT = 0;
    while (1) {
        OUT = !OUT;
        delay_ms(10U);
    }
}
```

Result. Only P2.4 changes; other latches retain their reset values. Nominal half-period=10 ms, full period=20 ms, about 50 Hz. Translated assembly uses a carry/bit operation; byte-level rewriting of unrelated bits is unnecessary.

8.8 C03: Sample P1, wait, then copy to P2

Problem. Copy a sampled input after nominal 500 ms. **Assumptions.** Classic 12 MHz; switches pull P1 inputs low; P1 latch is FFH. **Algorithm.** Capture a byte before the delay, then output that captured byte.

```
#include <reg51.h>
static void delay_ms(unsigned int ms)
{
    while (ms != 0U) {
        TR0 = 0;
        TMOD = (TMOD & 0xF0U) | 0x01U;
        TF0 = 0;
        TH0 = 0xFC; TL0 = 0x18;
        TR0 = 1;
        while (TF0 == 0) { }
        TR0 = 0;
        TF0 = 0;
        --ms;
    }
}
void main(void)
{
    unsigned char sample;
    EA = 0;
    P1 = 0xFF;
    P2 = 0x00;
    while (1) {
        sample = P1;
        delay_ms(500U);
        P2 = sample;
    }
}
```

Result / distinction. If P1 is A5H when sampled and changes during the delay, P2 still receives A5H. This is a delayed sample, not an instantaneous continuously refreshed copy. TMOD/TH0/TL0 are as in C01. P2 begins at 00H.

8.9 C04: Route a P0 input by its unsigned magnitude

Problem. If input is below decimal 100 (64H), send it to P1; otherwise send it to P2. **Assumptions.** P0 has external pull-ups and is released; no external memory bus. **Algorithm.** Read once into an unsigned byte and branch on the captured value.

```
#include <reg51.h>
void main(void)
{
    unsigned char sample;
    P0 = 0xFF;              /* release input pins */
    P1 = 0x00; P2 = 0x00;
    while (1) {
        sample = P0;
        if (sample < 100U) { P1 = sample; }
        else { P2 = sample; }
    }
}
```

Expected results. Input 42H=66 writes P1=42H; ABH=171 writes P2=ABH. The unselected output keeps its previous value, initially 00H. Equality with 100 belongs to the P2 branch. There is no delay requirement or undeclared helper.

8.10 C05: Send exactly the ASCII sequence 012345ABCD

Problem. Send ten characters on P1 without sending the string terminator. **Assumptions.** Classic 12 MHz; receiving device observes byte values, not a complete LCD protocol. **Algorithm.** Fetch CODE string bytes through the compiler, output indices 0–9, delay between writes, then stop.

```
#include <reg51.h>
unsigned char code sequence[] = "012345ABCD";
static void delay_ms(unsigned int ms)
{
    while (ms != 0U) {
        TR0 = 0;
        TMOD = (TMOD & 0xF0U) | 0x01U;
        TF0 = 0;
        TH0 = 0xFC; TL0 = 0x18;
        TR0 = 1;
        while (TF0 == 0) { }
        TR0 = 0;
        TF0 = 0;
        --ms;
    }
}
void main(void)
{
    unsigned char i;
    EA = 0;
    for (i = 0; i < sizeof(sequence) - 1U; ++i) {
        P1 = sequence[i];
        delay_ms(10U);
    }
    while (1) { }          /* final output is ASCII D */
}
```

Result. Bytes are 30H,31H,32H,33H,34H,35H,41H,42H,43H,44H. Array size is 11 including NUL; `sizeof(sequence)-1U` is 10. Final P1=44H. A loop through index 10 would also output 00H, an eleventh byte.

8.11 C06: Invert input P1.0 onto P2.7

Problem / algorithm. Release P1.0, read its pin, write the logical inverse to P2.7 repeatedly. A switch to ground is active low.

```
#include <reg51.h>
sbit INBIT = P1^0;
sbit OUTBIT = P2^7;
void main(void)
{
    INBIT = 1;             /* release input before reading */
    OUTBIT = 0;
    while (1) { OUTBIT = !INBIT; }
}
```

Result. Input 0 gives P2.7=1; input 1 gives P2.7=0. Other P2 bits are preserved. Use ! to express Boolean inversion clearly; the ^ declaration is not an executable XOR.

8.12 C07: Sensor-controlled square-wave driver signal

Problem. When P1.1 is high, generate an approximately 500 Hz driver signal on P1.7; when low, keep the driver inactive. **Assumptions.** Classic 12 MHz; active-high sensor and driver; external transistor/buzzer circuit, no direct high-current drive. A passive sounder requires the square wave; an active buzzer may only require an enable signal.

```
#include <reg51.h>
sbit SENSOR = P1^1;
sbit SOUND = P1^7;
static void half_period(void)
{
    TR0 = 0;
    TMOD = (TMOD & 0xF0U) | 0x01U;
    TF0 = 0;
    TH0 = 0xFC; TL0 = 0x18; /* 1 ms at 12 MHz */
    TR0 = 1;
    while (TF0 == 0) { }
    TR0 = 0; TF0 = 0;
}
void main(void)
{
    EA = 0;
    SENSOR = 1; SOUND = 0;
    while (1) {
        if (SENSOR) {
            SOUND = !SOUND;
            half_period();
        } else { SOUND = 0; }
    }
}
```

Derivation / behavior. One high or low interval is nominally 1000 ticks=1 ms; $f = 1/(2\times 1\,\text{ms}) = 500$ Hz before overhead. TH0:TL0=FC18H. Sensor low explicitly clears SOUND; response can wait until the current 1 ms interval completes. Contact bounce and sensor conditioning are circuit concerns. The slide's two 200 ms delays would give only 2.5 Hz if they truly were milliseconds; its undefined delay routine does not establish that timing.

Exam angle. Write a port-routing program, repair a missing input release, or derive half-period from a required output frequency.

9 Timer/counter hardware, registers and calculations

A timer increments on a clock derived from the oscillator. A counter increments on sampled external events. The same Timer 0/1 registers support both; TMOD selects the source, width and gating. Timers run while the CPU executes other instructions, so polling time overlaps counting.

9.1 Registers and control bits

Register	Byte address	Purpose
TL0 / TH0	8AH / 8CH	Timer 0 low / high count or reload byte.
TL1 / TH1	8BH / 8DH	Timer 1 low / high count or reload byte.
TMOD	89H	Two 4-bit configuration fields; not bit-addressable.
TCON	88H	Timer run/overflow bits plus external-interrupt control; bit-addressable.

7	6	5	4	3	2	1	0
GATE1	C/T1	M1	M0	GATE0	C/T0	M1	M0

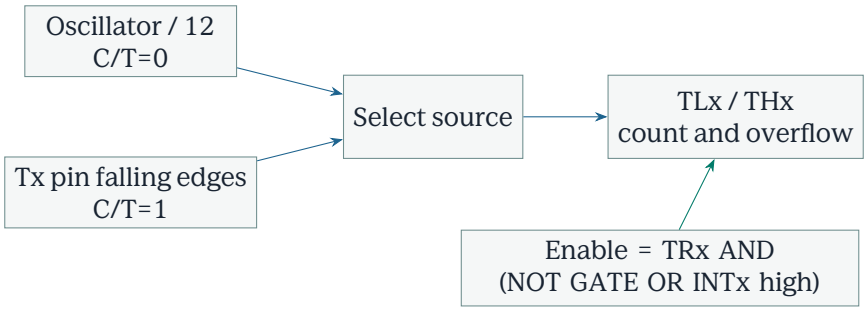
TMOD

Bits 7–4 configure Timer 1; bits 3–0 configure Timer 0. C/T=0 chooses timer ticks; C/T=1 chooses external counting. M1:M0 = 00,01,10,11 chooses mode 0,1,2,3. GATE=0 gives software enable via TR; GATE=1 additionally requires the corresponding INT pin high.

7	6	5	4	3	2	1	0
TF1	TR1	TF0	TR0	IE1	IT1	IE0	IT0

TCON

TR0 / TR1 start or stop counting; TF0 / TF1 record overflow and stay set until cleared. Polling programs clear them in software. Automatic flag clearing on interrupt entry is irrelevant here because interrupts remain disabled. IE/IT bits concern external interrupts; their full programming belongs to Module 3.



Thus GATE=0 does not mean “disabled”: counting runs when TR=1. GATE=1 with TR=0 never runs. With TR=1, INT high enables and INT low pauses the count. This can measure a high pulse, with input sampling/edge and software-read uncertainty.

Preserve the other timer

```
TMOD = (TMOD & 0xF0U) | 0x01U; /* T0 timer, mode 1, GATE=0 */
TMOD = (TMOD & 0x0FU) | 0x10U; /* T1 timer, mode 1, GATE=0 */
TMOD = (TMOD & 0x0FU) | 0x60U; /* T1 counter, mode 2 */
TMOD = (TMOD & 0xF0U) | 0x05U; /* T0 counter, mode 1 */
```

These are configuration illustrations. A full-byte assignment such as TMOD=01H is acceptable only when deliberately resetting Timer 1’s field too. Updating a mode while a timer runs can disrupt it:

stop the selected timer before reconfiguration. Each complete program below owns its selected timer.

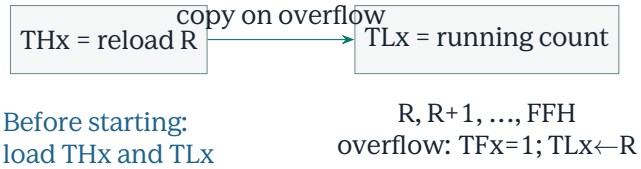
Exam angle. Decode 61H: Timer 1 is counter mode 2, GATE=0; Timer 0 is timer mode 1, GATE=0. *Source check: Class 6 pp. 1–12, Class 7 pp. 1–21, 43–46; Intel timer/counter section and manufacturer TMOD/TCON definitions..*

9.2 The four modes

Mode	Width	Counting and intended use
0 (00)	13 bits	THx supplies 8 bits and TLx low 5 bits supply 5; count $= 32\,THx + (TLx \,\&\, 1FH)$. Overflow after 8192 ticks from zero. Set unused TL upper bits to zero. Legacy compatibility.
1 (01)	16 bits	THx:TLx counts 0000H–FFFFH, then rolls to 0000H and sets TFx. Software must reload for a repeated chosen interval.
2 (10)	8 bits	TLx counts; on FFH-to-00H overflow, TFx is set and TLx automatically receives THx. THx remains the reload byte.
3 (11)	Split / hold	Timer 0: TL0 is an independent 8-bit timer/counter under TR0/TF0; TH0 is an 8-bit timer under TR1/TF1. Timer 1 placed in mode 3 holds its count. Timer 0 split mode appropriates Timer 1's ordinary run/flag controls.

Mode 3 is supporting classroom context, not the preferred solution to the required two-timer examples. Timer 1 can still have specialised uses such as baud-rate generation when Timer 0 is split; this guide does not develop serial configuration. Mode 0 maximum delay at 11.0592 MHz is $8192(12/11059200) = 8.888889$ ms; at 12 MHz, 8.192 ms.

Mode-2 first interval



Writing THx does *not* immediately load TLx. If TH1=05H but TL1 remains reset 00H, the first interval is 256 ticks; subsequent intervals are $256 - 5 = 251$ ticks. At 11.0592 MHz these are 277.777778 and 272.352431 μs. Initialize both for equal intervals. Leave the timer running for auto-reload; repeatedly stopping it adds dead time and phase shifts.

9.3 Derive the timing formulas

For the classic timer source, $f_t = f_{osc}/12$ and $t_t = 12/f_{osc}$. One *machine cycle* has twelve oscillator clocks. An instruction can consume more than one machine cycle; instruction length is a separate property.

Quantity	11.0592 MHz	12 MHz
Timer tick frequency	921600 Hz = 921.6 kHz	1000000 Hz = 1 MHz
Timer tick duration	$1.085069444\ldots$ μs	1.000000 μs
Mode-1 maximum, 65536 ticks	71.111111 ms	65.536000 ms
Mode-2 maximum, 256 ticks	277.777778 μs	256.000000 μs

For a b-bit count with preload R, count states are R through $2^b - 1$. The final increment to zero is included:

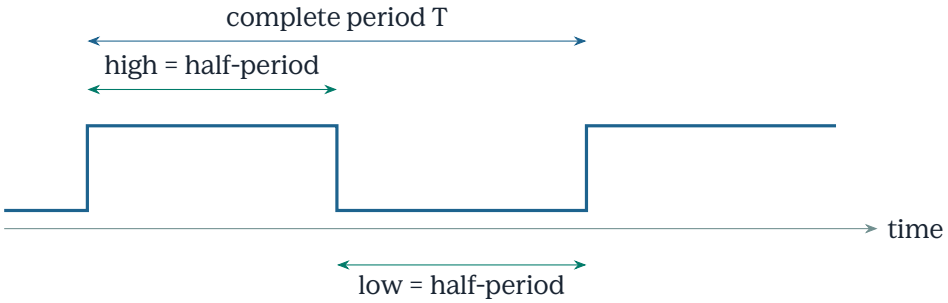
$$N = 2^b - R, \qquad d_h = Nt_t, \qquad R = 2^b - N.$$

For mode 1, b=16 and write TH=$\lfloor R/256 \rfloor$, TL=$R \bmod 256$. For mode 2, b=8 and write both TH and initial TL=R. For mode 0 use 8192 and split the 13-bit preload into high eight/low five bits.

For a desired delay d^*, use $N^* = d^*/t_t$ and choose a valid integer N. Nearest-integer rounding normally minimises delay error; round upward if a minimum delay is required. Then $d_a = Nt_t$ and delay error $100(d_a - d^*)/d^*$. For a toggled square wave:

$$d^*_{half} = \frac{1}{2f^*}, \qquad f_a = \frac{f_t}{2N}, \qquad e_f = 100\frac{f_a - f^*}{f^*}.$$

A pulse that only rises and falls once has a *pulse width*; it is not automatically a periodic waveform. Equal high/low intervals give a 50% duty-cycle square wave; unequal overhead on the two paths can change duty cycle.



Worked example: 50 Hz at 11.0592 MHz

Full period $T = 1/50 = 20$ ms; half-period=10 ms. $N = (0.010)(921600) = 9216 = 2400H$. Preload $65536 - 9216 = 56320 = DC00H$; therefore TH= DCH and TL=00H. Toggle once per overflow. A 50 kHz wave instead needs a 10 μs half-period, about 9.216 ticks, and software overhead becomes dominant. The slide's DC00H calculation supports 50 *Hz*, not 50 kHz.

Worked example: 500 Hz and quantisation

Desired half-period=1 ms; $N^* = 921.6$. Nearest N=922 gives R=FC66H, actual half-period $922/921600 = 1.000434028$ ms and $f_a = 499.783080$ Hz: frequency error -0.043384%. The classroom FC67H uses N=921, giving 0.999348958 ms and 500.325733 Hz, error $+0.065147\%$. Both are close before overhead; FC66H is closer under nearest-count rounding.

For 750 Hz, $N^* = 614.4$; choose N=614, R=FD9AH, half-period=666.232639 μs, $f_a = 750.488599$ Hz, error $+0.065147\%$. These are ideal timer-derived values; a stop/load/poll/toggle routine adds edge spacing.

9.4 Recomputed classroom preloads

All following mode-1 intervals use the unrounded 11.0592 MHz tick and exclude software overhead. Hex bases are explicit. The table counts rollover; using FFFFH minus preload alone loses one tick.

Preload	Decimal R	N	Hardware interval	Classroom context
FFF2H	65522	14	0.015190972 ms	C6 pp.10–14
B83EH	47166	18370	19.932725694 ms	C6 p.15
7634H	30260	35276	38.276909722 ms	C6 pp.16–17
0108H	264	65272	70.824652778 ms	C6 p.22
3500H	13568	51968	56.388888889 ms	C6 p.36

Preload	Decimal R	N	Hardware interval	Classroom context
4BFDH	19453	46083	50.003255208 ms	C6 p.38
A5FEH	42494	23042	25.002170139 ms	C6 p.39
EE00H	60928	4608	5.000000000 ms	C6 pp.18–19
DC00H	56320	9216	10.000000000 ms	C6 p.21
7748H	30536	35000	37.977430556 ms	C7 p.26
FF1AH	65306	230	0.249565972 ms	C6 p.20

Long-delay and frequency checks

- 0108H gives 65272 ticks = 70.824652778 ms; 200 intervals give 14.164930556 s. Multiplying an early 1.085 μs tick gives 14.164024 s instead.
- 4BFDH gives 46083 ticks = 50.003255208 ms; exact nominal 50 ms needs N=46080 and 4C00H.
- A5FEH gives 23042 ticks = 25.002170139 ms; twenty intervals give 500.043402778 ms. Exact nominal 25 ms uses A600H; twenty give 500 ms before gaps.
- EE00H gives 4608 ticks = 5 ms. A symmetric toggle gives T=10 ms and 100 Hz; one isolated high pulse has width 5 ms.
- With TH=00H in mode 2, each interval is 256 ticks. Accumulating 250 intervals for each half-period gives $T = 2(250)(256)/921600 = 0.138888889$ s and $f = 7.2$ Hz. The slide's 72 Hz is a factor-of-ten error.
- With TH=05H, steady N=251 and ideal toggle frequency $921600/(2 \times 251) = 1835.856574$ Hz. With TH=00H, ideal N=256 gives 1800 Hz. First TL initialization still matters.
- TH=-23 truncated to a byte is E9H=233, so *interval count*=23, not 233. It gives 24.956597222 μs. 9000 such intervals give 224.609375 ms; 10000 give 249.565972222 ms. Neither unexplained loop compensation nor stopping each time establishes exact 250 ms.
- TH=-184 becomes 48H, N=184; interval=199.652778 μs, ideal toggle frequency 2504.347826 Hz. At 12 MHz, N=200 and R=38H yield a 200 μs half-period and ideal 2500 Hz.

9.5 Hardware interval versus complete program delay

After TR is set, the timer counts while the CPU polls TF. Polling does not add an entire second interval: its instructions execute during the count. Detection waits until the CPU next tests the flag; subsequent clear, stop, reload and toggle instructions add further latency. Stopping mode 1 creates gaps between nominal count windows.

For an edge-to-edge claim, identify the exact instruction that writes the output, generated instruction timings, the timer's start/stop phase, polling-loop phase and all path differences. A slide's "N timer ticks plus every polling cycle" can double-count overlapping time. Here timer equations describe **ideal hardware intervals**; C-generated waveform periods are **approximate** unless a compiled edge trace is reported. Mode-2 free running removes reload gaps but still has polling jitter. TF is a sticky bit, not an overflow queue: if two overflows happen before servicing, software cannot infer two from one set flag.

9.6 DS89C420 is a different timing target

DS89C420 retains 8051 instruction compatibility but has a faster core; instruction clock counts differ from classic machine-cycle counts. Its timer clock selection is configurable. The manufacturer's CKCON/CKMOD fields select divide-by-12, divide-by-4 or system-clock operation for Timer 0/1, with clock-mode settings also relevant. Therefore oscillator/12 is a baseline assumption, not a rule for all derivatives. Recalculate both software loops and timer preloads from the chosen device's clock configuration. The classroom's fast-core delay cannot be copied as a classic delay. *Source check: Analog Devices / Maxim Ultra-High-Speed Flash Microcontroller User's Guide, revision 11, clock-control registers and Timer/Counters section. Use with the selected device data sheet..*

10 Complete Embedded C timer and counter programs

All these programs poll with interrupts disabled. Stop, configure, initialize count and flag, start, wait, then service. Preserve the unused timer's TMOD nibble. Mode-1 routines reload explicitly; mode-2 routines keep running and initialize both TH and TL.

10.1 C08: Timer 0 mode-1 50 ms hardware intervals

Problem / assumptions. Toggle P1.5 after each nominal 50 ms; classic 11.0592 MHz. **Algorithm.** Configure T0 mode 1; load 4C00H; poll TF0; stop and clear; toggle.

```
#include <reg51.h>
sbit OUT = P1^5;
static void delay50ms(void)
{
    TR0 = 0;
    TMOD = (TMOD & 0xF0U) | 0x01U;
    TF0 = 0;
    TH0 = 0x4C; TL0 = 0x00;
    TR0 = 1;
    while (TF0 == 0) { }
    TR0 = 0; TF0 = 0;
}
void main(void)
{
    EA = 0; OUT = 0;
    while (1) { delay50ms(); OUT = !OUT; }
}
```

Registers / result. TMOD low nibble=01H, TH0=4CH, TL0=00H. N=46080, hardware interval=50 ms; ideal full wave=100 ms, 10 Hz. The complete edge interval is longer because the routine stops/reloads. The slide's Timer 0 selection with Timer 1 register use is repaired consistently here.

10.2 C09: Timer 1 mode-1 repeated 25 ms intervals

Problem. Alternate P2 patterns every nominal 500 ms using twenty shorter intervals. **Assumptions.** Classic 11.0592 MHz; T1 reserved. **Algorithm.** Repeat a 25 ms count twenty times, then complement a software shadow byte and write it to P2.

```
#include <reg51.h>
static void delay25ms(void)
{
    TR1 = 0;
    TMOD = (TMOD & 0x0FU) | 0x10U;
    TF1 = 0;
    TH1 = 0xA6; TL1 = 0x00;
    TR1 = 1;
    while (TF1 == 0) { }
    TR1 = 0; TF1 = 0;
}
void main(void)
{
    unsigned char i;
    unsigned char pattern = 0x55;
    EA = 0;
    P2 = pattern;
    while (1) {
        for (i = 0; i < 20U; ++i) { delay25ms(); }
        pattern ^= 0xFFU;
        P2 = pattern;
    }
}
```

Registers / result. TMOD high nibble=10H, TH1=A6H, TL1=00H, N=23040. Twenty hardware windows sum to 500 ms; full pattern cycle approximately 1 s. Reload gaps add to this. The shadow variable changes the intended output without accidentally reading external port levels.

10.3 C10: Timer 1 mode-1 50 Hz square wave

Problem / assumptions. Generate approximately 50 Hz on P2.3; classic 11.0592 MHz. **Algorithm.** Use the derived DC00H half-period, clear TF1 each interval and toggle OUT.

```
#include <reg51.h>
sbit OUT = P2^3;
static void half_period(void)
{
    TR1 = 0;
    TMOD = (TMOD & 0x0FU) | 0x10U;
    TF1 = 0;
    TH1 = 0xDC; TL1 = 0x00;
    TR1 = 1;
    while (TF1 == 0) { }
    TR1 = 0; TF1 = 0;
}
void main(void)
{
    EA = 0; OUT = 0;
    while (1) { half_period(); OUT = !OUT; }
}
```

Registers / result. TMOD high nibble=10H; TH1=DCH, TL1=00H; 9216 ticks per nominal half-period. P2.3 alternates; simply clearing P2.3 repeatedly would not generate a waveform. Ideal 50 Hz becomes slightly slower with stop/reload/edge overhead.

10.4 C11: Timer 1 mode-2 auto-reload waveform

Problem / assumptions. Approximately 2.5 kHz on P2.7; **classic 12 MHz**. **Algorithm.** Load TH1=TL1=38H before starting; keep T1 running; clear each observed flag and toggle OUT.

```
#include <reg51.h>
sbit OUT = P2^7;
void main(void)
{
    EA = 0; OUT = 0; TR1 = 0;
    TMOD = (TMOD & 0x0FU) | 0x20U;
    TH1 = 0x38; TL1 = 0x38; /* both loaded for first interval */
    TF1 = 0; TR1 = 1;
    while (1) {
        while (TF1 == 0) { }
        TF1 = 0;
        OUT = !OUT;
    }
}
```

Registers / derivation. TMOD high nibble=20H; N=$256 - 56 = 200$. Each ideal interval=200 μs, full wave=400 μs, 2500 Hz. Initial TL1=38H gives the same first interval. Polling introduces edge jitter; service must finish before the next overflow. This fixes both the first-overflow problem and the temptation to stop a continuously reloading timer.

10.5 C12: A long delay from continuous mode-2 overflows

Problem. Toggle P1.5 every nominal 250 ms without empirical nested-loop compensation. **Assumptions.** Classic 11.0592 MHz; the flag is serviced within every 277.777778 μs interval. **Algorithm.** Keep T0 mode 2 free running and count 900 serviced overflows in a 16-bit variable.

```
#include <reg51.h>
sbit OUT = P1^5;
void main(void)
{
    unsigned int intervals;
    EA = 0; OUT = 0; TR0 = 0;
    TMOD = (TMOD & 0xF0U) | 0x02U;
    TH0 = 0x00; TL0 = 0x00;
    TF0 = 0; TR0 = 1;
    while (1) {
        for (intervals = 0; intervals < 900U; ++intervals) {
            while (TF0 == 0) { }
            TF0 = 0;
        }
        OUT = !OUT;
    }
}
```

Derivation / result. TH0=TL0=00H; $900(256)/921600 = 0.250$ s. Ideal full period=0.500 s, 2 Hz. Polling latency moves individual edges; interrupts, long added tasks or slow servicing can lose overflows. A 900-count loop needs `unsigned int`, not `unsigned char`. Zero reload is intentional and means the maximum 256 ticks, not zero delay.

10.6 C13: Switch-selectable 500 / 750 Hz signal

Problem. SW on P1.2 chooses the nominal frequency; output is P1.7. **Assumptions.** Classic 11.0592 MHz; SW is active-low and released, with a switch to ground. **Algorithm.** Read SW once per half-interval; select FC66H for 0 or FD9AH for 1; poll T0 mode 1; toggle OUT.

```
#include <reg51.h>
sbit SW = P1^2;
sbit OUT = P1^7;
static void half_period(unsigned char state)
{
    TR0 = 0;
    TMOD = (TMOD & 0xF0U) | 0x01U;
    TF0 = 0;
    if (state == 0U) { TH0 = 0xFC; TL0 = 0x66; }
    else { TH0 = 0xFD; TL0 = 0x9A; }
    TR0 = 1;
    while (TF0 == 0) { }
    TR0 = 0; TF0 = 0;
}
void main(void)
{
    EA = 0; SW = 1; OUT = 0;
    while (1) {
        half_period((unsigned char)SW);
        OUT = !OUT;
    }
}
```

Result. SW=0: ideal 499.783080 Hz; SW=1: ideal 750.488599 Hz; quantisation derived in Section 9.3. Software makes actual frequency lower. Switching mid-cycle can pair different high/low intervals; switch bounce can change selections. The pins now match the requested problem rather than the conflicting classroom declarations.

10.7 External event counters: required syllabus material

For C/T=1, Timer 0 uses **T0=P3.4**, Timer 1 uses **T1=P3.5**. On a standard 40-pin DIP these are pins 14 and 15. The separate gate inputs are INT0=P3.2 (pin 12) and INT1=P3.3 (pin 13). These package pin numbers do not apply to every package.

A classic counter recognises a sampled high-to-low transition. The input is sampled once per machine cycle; the high and low levels each need at least one machine cycle for reliable recognition. The maximum ideal rate is $f_{osc}/24$: 460800 events/s at 11.0592 MHz and 500000 events/s at 12 MHz. Clean levels and adequate pulse width are required; switch bounce can count multiple events. It is not a rising-edge counter. TR and GATE conditions still apply.

Keep the input latch 1, choose counter mode, clear TH/TL and TF, then run continuously when event preservation matters. Timer tick formulas do not convert external events into elapsed time: N=256-R or 65536-R now means *number of recognised falling edges* to overflow.

10.8 C14: Timer 1 mode-2 8-bit event display

Problem. Display a running modulo-256 event count on P1. **Assumptions.** T1=P3.5 receives conditioned falling-edge pulses; GATE=0; P1 drives eight current-limited indicators or buffered logic. **Algorithm.** Release T1; load zero in TH1 and TL1; count continuously; copy TL1 to P1.

```
#include <reg51.h>
sbit T1PIN = P3^5;
void main(void)
{
    EA = 0; TR1 = 0;
    T1PIN = 1;
    TMOD = (TMOD & 0x0FU) | 0x60U;
    TH1 = 0x00; TL1 = 0x00;
    TF1 = 0; TR1 = 1;
    while (1) {
        P1 = TL1;
        if (TF1) { TF1 = 0; }
    }
}
```

Registers / results. TMOD high nibble=60H (counter + mode 2). After 5 events TL1=05H and logical P1=05H. After 255 events TL1=FFH; event 256 sets TF1 and reloads TL1=00H; event 257 gives 01H. An active-low LED bank visually inverts the logical pattern; optionally write `P1 = (unsigned char)(TL1 ^ 0xFFU);` for active-low binary illumination. Flag clearing does not stop counting. Counts above 255 wrap; TF1 is not a multi-overflow total.

10.9 C15: Timer 0 mode-1 coherent 16-bit event display

Problem. Obtain a coherent running 16-bit count; send low byte to P1 and high byte to P2. **Assumptions.** T0=P3.4 has clean pulses, GATE=0. Fewer than 256 input events occur during one high–low–high read attempt; no interrupts disturb the bounded read. **Algorithm.** Read TH0, TL0, TH0 again; if the high bytes differ, retry; assemble the unsigned count and output it.

```
#include <reg51.h>
sbit T0PIN = P3^4;
static unsigned int read_count(void)
{
    unsigned char hi1, lo, hi2;
    do {
        hi1 = TH0;
        lo = TL0;
        hi2 = TH0;
    } while (hi1 != hi2);
    return ((unsigned int)hi1 << 8) | lo;
}
void main(void)
{
    unsigned int count;
    EA = 0; TR0 = 0; T0PIN = 1;
    TMOD = (TMOD & 0xF0U) | 0x05U;
    TH0 = 0; TL0 = 0; TF0 = 0; TR0 = 1;
    while (1) {
        count = read_count();
        P1 = (unsigned char)count;
        P2 = (unsigned char)(count >> 8);
        if (TF0) { TF0 = 0; }
    }
}
```

Why retry? Suppose count=12FFH. Reading high=12H, then rollover occurs before reading low=00H: a simple high–low read falsely returns 1200H. The second high=13H detects the mismatch. A retry yields 1300H or a later genuine sample. Stable high readings mean the low byte belonged to that high byte under the stated event-rate bound. Casting before the shift preserves the full unsigned value.

Registers / result. TMOD low nibble=05H; after 260 recognised events TH0=01H, TL0=04H, P2=01H, P1=04H. At 65536 events it wraps to 0000H. The captured 16-bit value is coherent; the two separate port writes are not an atomic 16-bit bus update. A downstream device needing simultaneous data must use a strobe/latch. This program counts modulo 65536, not an unlimited cumulative total.

10.10 C16: A deliberate stop-and-read contrast

Problem. Show why stopping a counter to read it can lose input events. **Assumptions.** Same T0 wiring as C15; this is supplementary diagnostic code, not the preferred continuous display. **Algorithm.** After an overflow, stop, read two bytes, output them, clear TF and restart.

```
#include <reg51.h>
sbit T0PIN = P3^4;
void main(void)
{
    unsigned char hi, lo;
    EA = 0; TR0 = 0; T0PIN = 1;
    TMOD = (TMOD & 0xF0U) | 0x05U;
    TH0 = 0; TL0 = 0; TF0 = 0; TR0 = 1;
    while (1) {
        if (TF0) {
            TR0 = 0;
            hi = TH0; lo = TL0;
            P2 = hi; P1 = lo;
            TF0 = 0; TR0 = 1;
        }
    }
}
```

Expected limitation. The stopped snapshot is internally consistent, but pulses while TR0=0 are not counted. Outputs update only after an overflow and show the post-rollover residue, not the total number of events. C15 avoids this deliberate stop gap. Neither example can recover missed pulses or infer multiple overflows from one sticky flag.

Exam angle. Choose timer versus counter mode, decode TMOD=60H, correct T0/INT0 pin confusion, or explain a torn multi-byte read.

Common mistake. Gate input INT0 is not the T0 event input. TH/TL must match the selected timer; setting TR0 cannot start Timer 1. *Source check: Class 6 pp. 44–45; Class 7 pp. 33–41, 45–46; Intel timer/counter operation and manufacturer register definitions..*

11 Answered examination practice

These are **authored preparation questions**, not predictions or an official marking scheme. Short answers are sized for approximately 3 marks. Long-answer choices follow the 9-mark full-question format with no more than three subdivisions. Read the question first, cover the answer, then compare your operands, flags, registers and units.

11.1 Short questions with concise answers

1. Distinguish immediate, direct and register-indirect addressing.
Immediate gives a literal: `MOV A,#30H` loads 30H. Direct names a byte address: `MOV A,30H` loads RAM[30H]. Indirect uses a pointer: after `MOV R0,#30H`, `MOV A,@R0` loads RAM[30H]. If RAM[30H]=A5H, results are 30H, A5H and A5H respectively.

2. Identify modes in MOV A,R3; MOV 40H,A; MOVC A,@A+DPTR.
R3 is a register source; 40H is a direct destination; MOVC uses indexed CODE addressing. The last instruction reads a code-table byte at DPTR+A; it does not read external data memory. One instruction can have different source/destination addressing modes.

3. Why are instruction size and timing different?
Size counts encoded code bytes; timing counts machine cycles. On classic hardware `MOV A,#55H` is 2 bytes/1 cycle, MOVC is 1 byte/2 cycles, MUL AB is 1 byte/4 cycles. At 12 MHz a cycle is 1 μs; at 11.0592 MHz it is 1.085069444 μs.

4. What do ORG, DB and END do?
ORG sets the assembler location counter in an absolute segment. DB emits constant bytes, including quoted character codes. END terminates assembly of that source; it does not generate a CPU halt. Put required routines before END and prevent execution from falling into table data.

5. Explain carry, overflow and parity.
CY represents an unsigned carry or SUBB borrow; OV reports signed arithmetic overflow. P is 1 when A has an odd number of set bits. For 7FH+01H=80H, CY=0, OV=1, P=1; AC=1 because FH+1 carries from bit 3. Unsigned and signed overflow need not agree.

6. State MUL AB and DIV AB results and exceptional cases.
MUL puts low product in A, high product in B; CY=0 and OV=1 if the product exceeds 255. DIV puts unsigned quotient in A, remainder in B, normally CY=OV=0. For B=0, CY=0, OV=1 and A/B results are undefined. Guard against a zero divisor rather than using those undefined bytes.

7. Why clear carry before a single-byte subtraction?
SUBB computes A minus source minus incoming CY. `CLR C` establishes no incoming borrow. Clear only before the low byte of multi-byte subtraction; keep its resulting borrow for the next byte. 25H–34H yields F1H with borrow 1; entering with CY=1 yields F0H.

8. When is DA A valid?
After ADD/ADDC of valid packed-BCD operands, DA A adjusts A and CY into a decimal result. It is not a general binary-to-BCD conversion and not a decimal-subtraction correction. 58H+67H followed by DA A yields A=25H, CY=1, representing decimal 125.

9. Distinguish byte address 20H and bit address 20H.
Byte 20H is one bit-addressable RAM byte. Bit address 20H maps to RAM byte 24H, bit 0. `MOV 20H,#01H` writes RAM[20H]; `SETB 20H` changes RAM[24H].0. Bit mapping is byte=20H+floor(bit/8), position=bit modulo 8.

10. State SJMP and AJMP restrictions.
SJMP has a signed offset -128 through $+127$ from the address *after* the instruction. AJMP is 2 bytes and chooses a destination in the same 2 KB page as that following instruction. ACALL has the same page rule; LJMP/LCALL can address the full 64 KB code space.

11. Describe stack operation for a call.
SP points to the last occupied internal RAM stack byte; push increments SP before storing. LCALL pushes return-PC low byte first, high byte second. RET pops high then low. At call address 0123H, return=0126H; with SP=60H, RAM61=26H and RAM62=01H; RET restores SP=60H.

12. Distinguish bit, sbit and sfr in C51.
`bit` allocates a one-bit Boolean object. `sbit OUT = P1^5;` aliases a bit address. `sfr` aliases an 8-bit SFR address; a standard header already declares P1. They are compiler extensions, not interchangeable ordinary integer types.

13. Why does a P0 input require pull-ups?
P0's ordinary GPIO output is open drain. A 0 drives low; a 1 releases the pin without the P1/P2/P3 internal pull-up. Write FFH to release an input byte and supply external pull-ups. Without them, a released input may float. External bus operation has different circuitry.

14. Explain port latch versus pin read.
A latch retains the programmed output; a pin reflects external voltage. `MOV A,P1` reads pins; `XRL P1,#0FFH` uses a latch read-modify-write. If an external switch holds a released bit low, a pin read sees zero while its latch remains one. Choose an operation matching the intended input/output behavior.

15. Decode TMOD=15H and state gate enable.
Timer 1 high nibble=1: timer mode 1, GATE=0. Timer 0 low nibble=5: counter mode 1, GATE=0. Each requires its TR bit. In general enable=TR AND (NOT GATE OR INT high); GATE=1 never bypasses TR.

16. Why load both TH and TL in mode 2?
TH stores the reload; TL is the live count. Loading TH alone does not copy it to TL immediately. TH=48H and TL=00H gives a first 256-count interval and later 184-count intervals. Load both with 48H before starting to obtain 184 counts from the first overflow onward.

17. What is maximum mode-1 delay at 12 MHz?
Tick=12/12000000=1 μs. Starting from zero requires 65536 ticks including rollover, giving 65.536 ms. A longer delay requires repeated windows or overflow accumulation. Using 65535 ticks omits the rollover increment.

18. State classic external-counter input and rate limits.
T0=P3.4 and T1=P3.5 count sampled falling edges. Inputs must have high/low durations at least one machine cycle each. Maximum ideal rate is oscillator/24; at 12 MHz it is 500 kHz. INT0=P3.2 and INT1=P3.3 are gate inputs, not the counted-event pins.

19. Why can a 16-bit counter read tear?
The CPU reads two bytes separately while hardware continues counting. A rollover between reads can mix old high with new low. Read high, low, high again and retry if high changes, under a bounded event rate. Stopping avoids a torn read but loses events during the stop.

20. Why is a written MSDelay(500) not proof of 500 ms?
A function name does not define its timing. The body may be missing, optimised away or dependent on generated instruction counts. A hardware-timer helper defines nominal count windows, but setup/poll/return overhead still affects total time. State compiler/clock evidence for precise software timing.

11.2 Operand and execution-trace drills

Drill A: values, addresses and bank selection. Assume bank 1, RAM[30H]=A5H, RAM[0DH]=77H, RAM[05H]=22H. Identify each result: `MOV A,#30H`; `MOV A,30H`; `MOV A,R5`; `MOV A,05H`.
Answer: 30H, A5H, 77H and 22H. R5 follows the selected bank; direct address 05H does not.

Drill B: carry trace. Start A=FFH, CY=0; execute `ADD A,#01H; MOV 30H,A; MOV A,#00H; ADDC A,#00H`.

Answer: First ADD gives A=00H,CY=1,AC=1,OV=0,P=0. RAM30=00H. MOV preserves CY. ADDC gives A=01H,CY=0,AC=0,OV=0,P=1. The two-byte sum is 0100H; read flags at the requested step, not only at the end.

Drill C: division base ambiguity. Compare division of literal 65 by literal 08 in A51 with 65H by 08H.
Answer: Bare 65 is decimal 65, so quotient=8 and remainder=1. 65H=101 decimal, so quotient=12=0CH and remainder=5. Classroom output 0CH/05H supports the explicitly labelled hexadecimal reconstruction.

Drill D: short jump. SJMP begins at 0100H and must reach 00F0H. What is the encoded offset?
Answer: Next instruction is 0102H. $00F0H - 0102H = -12H = -18$ decimal; 8-bit two's complement offset=EEH. Encoded bytes are 80H EEH. Measuring from 0100H gives an incorrect offset.

Drill E: the page boundary. An AJMP is at 07FEH. Can it target 0750H? Can it target 0850H?
Answer: Following address=0800H, whose 2 KB page spans 0800H–0FFFH. Target 0750H is outside; 0850H is inside. The page belongs to the following instruction, not simply the AJMP's starting byte.

Drill F: nested stack use. SP=60H; LCALL at 0123H; callee PUSH ACC with A=25H, writes A=55H, POP ACC, RET.
Answer: After call SP=62H, RAM61=26H, RAM62=01H. After push SP=63H, RAM63=25H. POP restores A=25H and SP=62H. RET restores PC=0126H and SP=60H. POP/RET do not erase the old RAM bytes.

11.3 Developed 9-mark choices: assembly and arithmetic

Choice A1 (9 marks): (a) explain memory spaces and transfer addressing (3); (b) develop a 17-byte internal-to-external block transfer (6).

Answer (a). Internal RAM uses MOV direct or @R0/@R1. CODE contains instructions/constant tables and uses MOVC for table reads. XDATA uses MOVX. `MOV A,#20H` gives a value; `MOV A,20H` reads internal address 20H; `MOV A,@R0` follows R0. Equal numeric addresses across these spaces do not alias.

Answer (b). Source 20H–30H inclusive has $30H - 20H + 1 = 11H = 17$ bytes; destination is XDATA 1020H–1030H. Require external RAM hardware and initialized source; forward overlap is irrelevant because the spaces differ. Establish bank 0, initialize source 00H–10H, set R0=20H, DPTR=1020H and R7=11H. In each iteration read @R0 into A, write MOVX @DPTR,A, increment both pointers, decrement/test R7. **Complete source: Program A04, page 12.** This reference forms part of the answer; its fully selectable listing includes all initialization and terminal labels. After 17 iterations R0=31H, DPTR=1031H, R7=0 and destination bytes=00H–10H. A count 0AH would copy only 20H–29H, not the stated inclusive range.

Choice A2 (9 marks): (a) distinguish CY/AC/OV/P (3); (b) develop 16-bit addition with carry (3); (c) develop subtraction with borrow (3).

Answer (a). Use the definitions and flag trace in Section 6: unsigned full-byte carry, nibble carry, signed overflow and odd accumulator parity. For 7FH+01H, CY=0,AC=1,OV=1,P=1; signed result overflows although unsigned result fits.

Answer (b). 1234H+00F0H: low 34H+F0H=124H, store 24H and retain CY=1. Load high 12H without changing CY, then ADDC 00H gives 13H; store it. Capture any carry beyond 16 bits with A=0 and ADDC 0. **Complete source: A08, page 16.** Result RAM30=24H,RAM31=13H,RAM32=00H, representing 1324H.

Answer (c). 1200H–00F1H: clear C before the low byte. Low subtraction gives 0FH,borrow=1; high SUBB gives $12H - 00H - 1 = 11H$,borrow=0. **Complete source: A10, page 17.** Result=110FH and final unsigned borrow=0. Do not clear C between bytes. These programs use byte storage conventions, independent of the high-first immediate encoding of DPTR/LCALL.

11.4 Developed 9-mark choices: Embedded C and I/O

Choice B1 (9 marks): (a) choose data types and declarations (3); (b) route a byte input conditionally with wiring (6).

Answer (a). Use unsigned char for an 8-bit port sample (0–255), unsigned int for 900 counts (0–65535), sbit for one hardware pin and bit for an allocated Boolean. In a declaration `sbit SW=P1^2;` selects bit 2; in an expression caret is XOR. CODE strings use the code qualifier.

Answer (b). Release P0 with FFH and external pull-ups. Use active-low switches or a logic source with correct levels. Initialize P1/P2, sample P0 once, branch on `sample < 100U`, write the selected port and retain the unselected output. **Complete source: C04, page 29.** Input 63H=99 goes to P1; 64H=100 goes to P2. Add one resistor per indicator and a buffer if the selected device's aggregate current limit requires it. No unknown delay function is needed.

Choice B2 (9 marks): (a) explain input release and read-modify-write (3); (b) develop a corrected ten-character ASCII output (3); (c) explain a sensor-controlled waveform (3).

Answer (a). Write 1 to a quasi-bidirectional input latch before reading; read the pin. A latch read-modify-write preserves unrelated intended output bits; an ordinary pin read can see external lows. P0 additionally needs external pull-ups. `CPL P1` targets a bit-address interpretation or is rejected; whole-port latch complement is `XRL P1,#0FFH`.

Answer (b). String "012345ABCD" contains ten characters and a C NUL terminator. Use `i < sizeof(sequence)-1U`, not `i <= 10`. **Complete source: C05, page 29.** Outputs are 30H–35H then 41H–44H, ending at D. The listing defines its delay helper; no LCD initialization is implied.

Answer (c). For nominal 500 Hz, half-period=$1/(2 \times 500) = 1$ ms. At 12 MHz, N=1000 and preload FC18H. Sample the released sensor, toggle a driver signal and wait one half-interval when active; explicitly clear it when inactive. **Complete source: C07, page 30.** Use a transistor/load driver; actual frequency is approximate because software adds time.

11.5 Developed 9-mark choices: timers

Choice C1 (9 marks): (a) derive a 50 Hz preload (3); (b) develop Timer 1 mode-1 C (3); (c) diagnose the classroom errors (3).

Answer (a). At 11.0592 MHz, tick=$12/11059200$ s, full period=20 ms and half-period=10 ms. N=9216; R=65536–9216=DC00H. TH1=DCH,TL1=00H; ideal delay quantisation error=0.

Answer (b). Preserve TMOD's low nibble, set high nibble 10H, stop T1, clear TF1, load TH1/TL1, start TR1, poll, stop, clear and toggle P2.3. **Complete source: C10, page 37.** TCON controls TR1/TF1; TMOD is not bit-addressable. Actual 50 Hz is approximate because setup and servicing add edge time.

Answer (c). Class 6 p.21 says 50 kHz but uses a 10 ms half-period, so the math is for 50 Hz. CLR on the output never alternates it. Not clearing TF1 makes later polling return immediately. Use OUT=!OUT and explicit per-interval flag service.

Choice C2 (9 marks): (a) explain mode-2 reload with a diagram (3); (b) derive a 2.5 kHz wave at 12 MHz (3); (c) develop a 250 ms repeated-overflow delay at 11.0592 MHz (3).

Answer (a). TH holds the reload, TL counts. At overflow TL is reloaded and TF set; TH is unchanged. The vector diagram in Section 9.2, page 32, is part of this answer. Initialize TH and TL together before TR=1; loading TH alone gives a possibly different first interval.

Answer (b). Half-period=200 μs; at 12 MHz N=200, R=256–200=56=38H. T1 mode 2 needs TMOD high nibble 20H. TH1=TL1=38H. **Complete source: C11, page 37.** Keep running; poll/clear TF1 and toggle P2.7. Ideal full period400 μs; service adds jitter.

Answer (c). T0 mode 2 with TH0=TL0=00H gives 256 ticks at 11.0592 MHz. $0.250 \times 921600/256 = 900$ intervals. Use unsigned int, clear each observed overflow and toggle after 900. **Complete source:**

C12, page 38. A second overflow before flag service is lost; continuous counting requires bounded service time.

11.6 Developed 9-mark choices: counters

Choice D1 (9 marks): (a) compare timer and counter clocks / pins (3); (b) develop an 8-bit external event counter (6).

Answer (a). Timer C/T=0 uses oscillator/12 on the classic baseline. Counter C/T=1 uses sampled falling edges on T0=P3.4 or T1=P3.5. INT0/INT1 are separate gate pins. TR must be set; GATE=1 additionally requires INT high. Maximum ideal count rate=oscillator/24 with sufficient high/low pulse widths.

Answer (b). Set T1 latch high, stop T1, preserve TMOD low nibble and set high 60H. Load TH1=TL1=0, clear TF1, start TR1, then display TL1 continuously on P1. **Complete source: C14, page 40.** Event 256 rolls FFH to 00H and sets TF1; event 257 gives 01H. Use conditioned pulse inputs and resistors/buffering for the display. Active-low LED illumination inverts the logical bits. Leaving the counter running avoids a deliberate stop gap.

Choice D2 (9 marks): (a) explain a torn read (3); (b) develop a coherent 16-bit count (3); (c) explain overflow/event loss (3).

Answer (a). With count 12FFH, high read 12H before rollover and low read 00H after rollover falsely combine as 1200H. Hardware changes independently of the CPU's two byte reads.

Answer (b). Use T0 mode 1 counter, TMOD low 05H, T0 input P3.4 released. Read high–low–high and retry on high mismatch. Cast high to unsigned int before shifting. **Complete source: C15, page 41.** Under the bounded read-rate assumption, TH=01H,TL=04H after 260 events yields P2=01H,P1=04H. Separate display writes may still need a downstream strobe.

Answer (c). Sixteen-bit count wraps every 65536 events. TF records that at least one overflow happened, not how many. Stop/read/restart loses events during the stop (C16). Overly short pulses or excessive input frequency are also missed. A wider total requires timely overflow accounting and a larger counter design beyond this simple modulo display.

11.7 Modified calculations and debugging

1. Modified clock: derive 400 Hz at 12 MHz, mode 1.
Half-period=$1/800 = 1.25$ ms; tick=1 μs, N=1250=04E2H, R=FB1EH. TH=FBH,TL=1EH; ideal frequency exactly400 Hz and quantisation error0. Configure one timer consistently and toggle once per overflow; account separately for software overhead.

2. Modified mode 2: 100 μs at 11.0592 MHz.
$N^* = 92.16$, choose 92; R=164=A4H. Actual interval=$92/921600 = 99.826389\,\mu$s; delay error=$-0.173611\%$. Ideal toggle frequency=5008.695652 Hz, versus desired5000 Hz; frequency error=$+0.173913\%$. Load TH=TL=A4H.

3. Modified long delay: 100 ms at 12 MHz.
A single mode-1 window can provide at most65.536 ms. Use two nominal 50 ms intervals: N=50000=C350H, preload 3CB0H. Two hardware intervals sum100 ms; stop/reload overhead adds time. A mode-2 alternative uses a suitable integer number of shorter windows with service-time checks.

4. Debug the block loop: range 20H–30H, count 0AH.
Inclusive length=$30H - 20H + 1 = 11H = 17$. Count0AH copies10 bytes, ending source 29H. Correct count to 11H or explicitly change the requested range. Pointers end one past the last byte, so correct final R0=31H.

5. Debug TMOD=01H followed by TH1/TL1/TR1/TF1.
The low 01H field configures T0 while the registers operate T1. Either use T0 throughout or configure

T1 high 10H while preserving low nibble. Also replace a repeated output clear with a toggle when a waveform is requested.

6. Debug TH1=05H, reset TL1=00H, TR1=1.
First interval 256 ticks, later 251 ticks; at 11.0592 MHz: 277.777778 and 272.352431 μs. Load TL1=05H too. If stopped/restarted each iteration, add dead time instead of claiming the ideal free-running frequency.

7. Debug END before DELAY.
The assembler terminates before defining DELAY, leaving a referenced routine unresolved or excluded. Move END after all code and tables; ensure the main path loops or jumps over helpers/data. END is not an executed stop.

8. Debug signed representation and string bounds.
FFH as a signed byte is-1, not-127. A ten-character C string occupies11 bytes including NUL; looping 0 through 10 sends 11 bytes. Use an explicitly signed type and a length bound that excludes the terminator when required.

12 Final revision and reading route

12.1 Compact revision sheet

Remember	Check before answering
Literal / address	`#30H` is a value; `30H` is direct; `@R0` dereferences.
Memory spaces	MOV internal/SFR; MOVC CODE; MOVX XDATA.
Registers	A/B/Rn/SP/PSW are 8-bit; PC/DPTR are 16-bit.
Arithmetic	ADD ignores incoming CY; ADDC/SUBB use it; clear initial subtraction borrow.
Flags	CY unsigned; OV signed; AC nibble; P odd set-bit count of A.
MUL / DIV	Product B:A; quotient A/remainder B; guard B=0; DA only packed-BCD addition.
Loop / call	DJNZ tests its decremented operand; JNZ tests A; RET pops return PC.
Stack	Push increments SP; call pushes low return byte first; reserve safe RAM.
C	char 8 / int 16 / long 32 in C51; specify unsigned; bit/sbit/sfr differ.
Ports	Release inputs with 1; P0 pull-ups; latch RMW differs from pin read.
TMOD	High nibble T1, low nibble T0; GATE,C/T,M1,M0; preserve the other field.
TCON	TF1/TR1/TF0/TR0 are bits 7/6/5/4.
Timer	Classic $f_t = f_{osc}/12$; N=$2^b - R$ includes rollover.
Wave	Half-period=$1/(2f)$; period includes high and low.
Mode 2	TH reload, TL count; initial TH=TL; keep running; service every overflow.
Counter	T0 P3.4/T1 P3.5 falling edges; INT0 P3.2/INT1 P3.3 gates; maximum oscillator/24.
Timing claim	Unrounded tick; quantisation; software overhead/phase; no exact empty-loop claim.

12.2 Syllabus completion checklist

You should be able to: explain source-to-code conversion; represent unsigned/signed/BCD/ASCII data; use ORG/EQU/DB/END; identify every required addressing mode; trace transfer and arithmetic routines; propagate carry/borrow; follow a call's stack; select C51 data types; release/read/drive ports; calculate and configure both timer modes; initialize auto-reload's first interval; and implement required external event counting with coherent display.

Use the coverage map on page 4 to find the teaching sections. If a checklist item remains difficult, trace its complete program before memorising a final number. For time management, a 9-mark answer should state assumptions, derive or explain configuration, give a consistent program and describe the result; labels and units count toward clarity.

12.3 Prescribed books: verified bibliography and chapter map

The syllabus's author spelling is imperfect. Verified catalog details are:

- Muhammad Ali Mazidi, Janice Gillispie Mazidi and Rolin D. McKinlay, *The 8051 Microcontroller and Embedded Systems: Using Assembly and C*, second edition, Pearson India, 2007, ISBN 9788131710265.

- Kenneth J. Ayala, *The 8051 Microcontroller*, third edition, Cengage India listing, 2007, ISBN 9788131502006.

Study topic	Mazidi, second edition	Ayala, third edition
Architecture prerequisite	Ch. 1, *The 8051 Microcontrollers*	Ch. 3, *The 8051 Architecture*
Assembly / directives	Ch. 2, *8051 Assembly Language Programming*	Ch. 4, *Basic Assembly Language Programming Concepts*; App. B, *Use of the A51 Assembler*
Addressing / transfer	Ch. 5, *8051 Addressing Modes*; Ch. 14 for supplementary external-memory detail	Ch. 5, *Moving Data: Logical Operations*
Arithmetic / flags	Ch. 6, *Arithmetic, Logic Instructions, and Programs*	Ch. 2, *Numbering Systems and Binary Arithmetic*; Ch. 6, *Arithmetic Operations*
Loops / calls	Ch. 3, *Jump, Loop, and Call Instructions*	Ch. 7, *Jump and Call Instructions*
I/O / C	Ch. 4, *I/O Port Programming*; Ch. 7, *8051 Programming in C*	Ch. 8, *An 8051 Microcontroller Design*; Ch. 9, *Applications*, for hardware/application context
Timer / counter	Ch. 9, *8051 Timer Programming in Assembly and C*; C timer/counter section 9.3	Ch. 3 architecture/control registers; Ch. 9 applications; App. F, *Control Registers*

Access boundary. Publisher bibliographic records and contents were accessible; the full prescribed textbook chapter bodies were not available for consultation in this run. This is a reading map, not a claim that those chapters verified individual answers. No invented book quotations or page references are used. C51 examples and calculations were checked using accessible primary documentation and translated tools described in the verification record.

13 Supporting classroom instruction material

The official ALP scope is data transfer and arithmetic. The following compact reference retains useful logical, bit and control material without treating it as a second full instruction-set course. It supports port masks, tests and loop/call routines already taught.

Family / example	Effect	Limits / flags
ANL A,#0FH	Keep the low nibble; e.g. A=A5H becomes 05H.	AND, not arithmetic multiplication. CY unchanged; P follows A.
ORL A,#80H	Set bit 7; A=25H becomes A5H.	OR, not arithmetic addition with carry.
XRL A,#0FFH	Complement eight bits; A=55H becomes AAH.	Mnemonic XRL, not XORL.
ANL/ORL/XRL direct,#data	Byte mask applied to a direct destination.	3 bytes/2 cycles; a port destination uses its latch.
CLR A; CPL A	Clear/complement the accumulator byte.	Both 1 byte/1 cycle. No generic CPL byte-register instruction.
SETB bit; CLR bit; CPL bit	Set, clear or complement one addressed bit.	2 bytes/1 cycle; SETB C / CLR C / CPL C are 1 byte/1 cycle.
ANL C,bit / C,/bit	Carry AND bit (or complemented bit).	2 bytes/2 cycles. CY is Boolean working storage.
ORL C,bit / C,/bit	Carry OR bit (or complemented bit).	2 bytes/2 cycles; syntax /bit complements the tested value.
MOV C,bit; MOV bit,C	Copy one Boolean bit.	Both 2 bytes; respectively 1 and 2 cycles.
RL A; RR A	Rotate accumulator without CY. A=81H: RL gives 03H; RR gives C0H.	Bit rotated out re-enters A; CY unchanged.
RLC A; RRC A	Rotate through CY. A=81H,CY=0: RLC gives 02H,CY=1; RRC gives 40H,CY=1.	These examples start independently from 81H/CY0; P follows new A.
SWAP A	Exchange high/low nibbles; A=3CH becomes C3H.	CY unchanged; nibble exchange is not bit reversal.
XCH A,direct / Rn / @Ri	Swap an entire byte with A.	Other operand forms must be supported; cannot XCH two arbitrary direct bytes without A.
XCHD A,@Ri	Swap only low nibbles. If A=ABH and RAM[30H]=CDH, result A=ADH, RAM[30H]=CBH.	R0/R1 pointer, unchanged high nibbles;1 byte/1 cycle.
JMP @A+DPTR	Set PC to the sum DPTR+A.	Computed CODE jump, not MOVC table-data read;1 byte/2 cycles.

Source check: Class 1 pp.61–62,73–94,95–112; Keil opcode table and individual instruction definitions..

13.1 Classroom signal examples with later-module peripherals

Class 5 p.19 writes message bytes on P1 and pulses an LCD enable pin P2.0. The intended text “The Earth is but One Country” has **28 characters**; a C string also has a terminator. A loop `z < 28` is consistent *if those exact spaces are present*. The scan/OCR can merge “but One” into “butOne”; this is transcription uncertainty, not enough evidence to declare the intended 28-character count wrong. A robust source uses `sizeof(message)-1U` instead of a handwritten length.

The relevant Module 2 idea is data output plus a control strobe; A17 demonstrates that concept with a complete signal program. A complete LCD driver additionally needs command/data selection, read/write control, initialization, device timing and busy/delay handling. None is implied by the classroom fragment. Class 6 p.46 is a serial-communication divider/cover; full UART programming stays outside this module. Later ADC/DAC/motor/interrupt drivers are similarly excluded.

14 Classroom discrepancies, recovery and verification

C1–C7 below mean the seven named decks in the source inventory. Every page reference is a **physical PDF page**, not a slide's embedded page label. Categories: **E** verified error in teaching content; **P** missing prerequisite or incomplete example; **S** intentional simplification needing a boundary; **R** labelled reconstruction or transcription uncertainty; **V** a suspected error not confirmed by visual inspection. Corrections use primary sources R1–R8 listed later and the independent calculations/program checks.

14.1 Discrepancy appendix

Slide	Kind	Finding and corrected teaching	Check / guide
C1 p.8	E	MOV A,22H denotes direct data; an immediate comment requires #22H. Value and address must be separated.	R1/R3; 4
C1 p.9	E	MUL A,B is malformed; standard syntax is MUL AB.	R1/R3; A11
C1 p.22	E/P	EQU takes an expression without immediate #. Use BIT for an explicitly named bit alias when needed; a declaration does not initialize it.	R4; 3.4
C1 p.26	E	"All registers are 8-bit" overlooks PC and DPTR (16 bits).	R1; 2
C1 pp.30–34	E	General length rules fail; use the full operand form. DPTR/LJMP/LCALL 16-bit immediates encode high byte before low.	R2/R3; 3
C1 pp.52, 65, 80–82, 89, 102	E	Correct MOCX→MOVX, AAD→ADD, XORL→XRL, SET→SETB and RET1→RETI.	R2/R3; 13
C1 pp.54–60	E/R	Rx/Ry is ambiguous direct-address notation, not a generic Rn-to-Rm opcode. POP copies stack[SP] to its direct operand, then decrements SP; it does not necessarily copy A.	R1/R3; 5,7
C1 pp.67–68	E	SUBB subtracts current CY, not an unconditional 1. @Ri denotes pointed RAM, not the numeric pointer byte.	R3; 6
C1 pp.70–71	E	INC/DEC @Ri changes pointed RAM. INC/DEC Ri changes the pointer. Ordinary byte INC/DEC leaves CY unchanged.	R3; A13
C1 pp.74,78–79	E	CPL A is 1 byte; ORL A,@Ri is 1 byte; ORL direct,#data is 3 bytes.	R2; 13
C1 pp.86–89	E	Boolean carry ANL/ORL take2 cycles. MOV C,bit takes1 cycle; MOV bit,C takes2. Check operand-specific tables.	R2; 13
C1 pp.96,107	V	Enlarged pages express JNZ as A≠0. This is correct; no contradictory JNZ condition was confirmed there. Do not confuse JNZ with DJNZ.	R3; 7
C1 p.102	E	AJMP is 2 bytes, not1. Target shares the following instruction's2KB page.	R2/R3; 7

Slide	Kind	Finding and corrected teaching	Check / guide
C1 pp.109,112	E/R	CJNE heading/operation mixes @Ri and Rn; DJNZ description must decrement/test the actual Rn or direct operand.	R3; 7
C2 p.3	E	Addition bytes end at address 0BH; location counter after them is 0CH, not the listing's END address 0AH. END emits no bytes.	Assembled A06
C2 pp.8–10	E/R	Make ORG/DPTR values 4500H/4100H explicit; a nonzero ORG needs a launch arrangement. Multiply 0AH by 88H gives 0550H. Division65H/08H gives 0CH/05H; base reconstructed from outputs. Bottom division inputs copied 0AH/88H are not consistent with that division.	R3; A07/A11/A12
C2 pp.11–12	P/E	Initialize block data and define terminal loop. 20H–30H inclusive is 17 bytes; count 0AH covers only 10. Exchange blocks 30H–34H/40H–44H has5 bytes.	A04/A05
C3 pp.5,8,13	E	SJMP range is-128 to$+127$, not symmetric± 127. DJNZ tests its decremented operand. Move END after DELAY so the label is assembled.	R3/R4; 7
C4 pp.7,9–10	E/P	P0 output needs pull-ups and a defined helper. CPL P1 cannot complement a whole byte; use XRL P1,#0FFH. Supply the missing SJMP target. Delay count 100 gives 203 cycles inside routine, 205 with LCALL on classic core.	R1/R3/R6; A15/A16
C4 pp.15–16	P/S	Input/strobe/oven examples need a terminal path and input release. An output pulse is not a complete audible buzzer waveform or load driver.	A17; 8.5
C5 pp.7,10–12	E	Signed integers use two's complement, not sign/magnitude. sbit is a single-bit alias, bit a Boolean object, sfr an SFR alias; SFR space is not ordinary upper RAM.	R5; 8
C5 pp.13–17	P/E	Undefined/empty MSDelay does not define milliseconds. "012345ABCD" has 10 characters; loop through 10 includes NUL. Release P0 and supply pull-ups. A delayed sample is not live copying.	R5; C01–C05
C5 p.18	E/P	200 ms high+200 ms low would mean2.5 Hz, not a few hundred Hz, if calibrated. Define1 ms half interval for nominal 500 Hz and an inactive output state; provide a load driver.	C07; 9.3
C5 pp.19–21	S/R	LCD example is only data/strobe; intended28-character phrase depends on spaces recovered from the image. Release input bits; distinguish Boolean ! from promoted bytewise complement.	R7; 13,C06
C6 p.6	E	11.0592 MHz/12=921.6 kHz, not921.6 MHz. Tick is 1.085069444 μs.	R1; 9.3

Slide	Kind	Finding and corrected teaching	Check / guide
C6 p.14	S/E	Timer counting overlaps the polling loop. A proposed exact total cannot add every poll cycle to the nominal interval without phase analysis.	R1; 9.3
C6 p.20	E	TMOD01H chooses T0 but code uses T1 registers/bits. Repeated CLR gives no toggle; select one timer consistently and complement output.	C08/C10
C6 p.21	E	Heading 50 kHz conflicts with 50 Hz math/DC00H. Clear TF1 each polling interval and toggle the output.	C10
C6 pp.23–26	P/E	TH-only load leaves a different first interval; load TL too. Auto-reload is intended to run continuously. Stopping each interval adds gaps.	R1; C11
C6 p.27	E	Ideal period 138.888889 ms yields 7.2 Hz, not 72 Hz. This omits stop/restart/software overhead.	9.3
C6 p.37	E	Oscillator/12 is not universal across8051 derivatives. DS89C420 clock configuration must be checked separately.	R6; 9.3
C6 pp.38–39	S	4BFDH/A5FEH are near 50/25 ms, not exact. Use 4C00H/A600H for exact nominal classic windows; repeated software adds gaps.	C08/C09
C6 pp.40–41	E	Requested SW=P1.2 and OUT=P1.7 conflict with declarations SW=P1.7/OUT=P1.5. C13 follows the requested pins; nearest500 Hz reload is FC66H.	C13
C6 pp.42–43	P/S	Initialize both mode 2 bytes. -23/-184 are byte reload representations, not interval counts 233/72. Unexplained 250×36 versus 250×40 compensation and stopping do not prove an exact delay.	C11/C12
C6 pp.44–45	P	Counter input aliases/release and TL initialization must be explicit. Two-byte reads can tear; stop/read loses events.	R1; C14–C16
C7 pp.3,22,26	V/S	Divider-to-square-wave illustration is conceptual, not automatic GPIO output. Enlarged p.22/p.26 useμs, notms: the suspected unit typo was not confirmed. For 35000 ticks, interval=37.977430556 ms; early 1.085 gives 37.975 ms.	9.3
C7 pp.31,33–41	P/S	Counter material is mandatory despite “information only”. Mode 2 needs firstTL load and bounded flag service; minimum ideal toggle frequency with 256 ticks is 1800 Hz.	C11/C14/C15

Slide	Kind	Finding and corrected teaching	Check / guide
C7 p.45	E	T0 counter is P3.4 (DIP 14), not P3.2. INT0 gate is P3.2 (DIP 12). C7 p.46 T1 P3.5/INT1 P3.3 separation is consistent.	R1; 10.7

14.2 Page-by-page source inventory and recovery map

ID	Original filename (preserved)	Pages
C1	`Module2-Class1_ALPFinal.pptx (1).pdf`	113
C2	`Modue2_Class2-ALPExamples.pdf`	12
C3	`Module2_Class3_CallandJump.pdf`	14
C4	`Module2-Class4_PortProgramming (1).pdf`	16
C5	`Module2-Class5_EmbeddedC.pdf`	21
C6	`Module2_Class6_TimerProgramming.pdf`	46
C7	`Module2_Class7MICROCONTROLLER TIMERSFinal.pdf`	47

All 269 pages were text-extracted and rendered. Every page was inspected in contact sheets; image-heavy code, symbols and diagrams were enlarged. OCR supplemented the image pages and was compared with those renders. No screenshot appears as teaching material in this guide: meaningful examples are selectable text; the memory, PSW, stack, ports and timer diagrams are redrawn vectors. Repeated instruction tables are consolidated into corrected operand-specific references.

The following inventory gives one row per physical source page. **T** means selectable teaching text/vector content; **I** means substantive teaching is a bitmap (searchable title alone is excluded); **M** means both. Vector diagrams count as T and were still visually inspected. Covers/syllabus/dividers map to scope rather than being treated as omitted teaching. Destination references point to the consolidated explanation, program or explicitly bounded supplementary material.

Source page	Kind	Recovered topic	Destination / page
C1 p.1	T	Cover and syllabus	1 / 4
C1 p.2	T	Cover and syllabus	1 / 4
C1 p.3	T	Assembly motivation	3 / 7
C1 p.4	T	Assembly motivation	3 / 7
C1 p.5	T	Statement, label, operand, comment	3 / 7
C1 p.6	I	Statement, label, operand, comment	3 / 7
C1 p.7	T	Statement, label, operand, comment	3 / 7
C1 p.8	I	Statement, label, operand, comment	3 / 7
C1 p.9	T	Statement, label, operand, comment	3 / 7
C1 p.10	T	Statement, label, operand, comment	3 / 7
C1 p.11	I	Source, object, listing, HEX workflow	3 / 7
C1 p.12	I	Source, object, listing, HEX workflow	3 / 7
C1 p.13	T	Source, object, listing, HEX workflow	3 / 7
C1 p.14	T	Source, object, listing, HEX workflow	3 / 7
C1 p.15	T	Source, object, listing, HEX workflow	3 / 7
C1 p.16	T	Source, object, listing, HEX workflow	3 / 7
C1 p.17	T	Source, object, listing, HEX workflow	3 / 7
C1 p.18	T	Source, object, listing, HEX workflow	3 / 7
C1 p.19	T	Assembler directives	3.4 / 8
C1 p.20	T	Assembler directives	3.4 / 8

Source page	Kind	Recovered topic	Destination / page
C1 p.21	T	Assembler directives	3.4 / 8
C1 p.22	T	Assembler directives	3.4 / 8
C1 p.23	T	Assembler directives	3.4 / 8
C1 p.24	T	Assembler directives	3.4 / 8
C1 p.25	T	Instruction/data/register introduction	3.3 / 7
C1 p.26	T	Instruction/data/register introduction	3.3 / 7
C1 p.27	T	Instruction/data/register introduction	3.3 / 7
C1 p.28	T	Instruction/data/register introduction	3.3 / 7
C1 p.29	T	Instruction/data/register introduction	3.3 / 7
C1 p.30	T	Instruction length and byte encoding	3 / 7
C1 p.31	T	Instruction length and byte encoding	3 / 7
C1 p.32	T	Instruction length and byte encoding	3 / 7
C1 p.33	T	Instruction length and byte encoding	3 / 7
C1 p.34	T	Instruction length and byte encoding	3 / 7
C1 p.35	T	Addressing mode definitions/examples	4 / 10
C1 p.36	T	Addressing mode definitions/examples	4 / 10
C1 p.37	T	Addressing mode definitions/examples	4 / 10
C1 p.38	T	Addressing mode definitions/examples	4 / 10
C1 p.39	T	Addressing mode definitions/examples	4 / 10
C1 p.40	T	Addressing mode definitions/examples	4 / 10
C1 p.41	T	Addressing mode definitions/examples	4 / 10
C1 p.42	T	Addressing mode definitions/examples	4 / 10
C1 p.43	T	Addressing mode definitions/examples	4 / 10
C1 p.44	T	Addressing mode definitions/examples	4 / 10
C1 p.45	T	Addressing mode definitions/examples	4 / 10
C1 p.46	T	Addressing mode definitions/examples	4 / 10
C1 p.47	T	Instruction family/image overview	13 / 51
C1 p.48	I	Instruction family/image overview	13 / 51
C1 p.49	I	Instruction family/image overview	13 / 51
C1 p.50	I	Instruction family/image overview	13 / 51
C1 p.51	T	MOV/MOVC/MOVX, PUSH/POP	5 / 11
C1 p.52	I	MOV/MOVC/MOVX, PUSH/POP	5 / 11
C1 p.53	T	MOV/MOVC/MOVX, PUSH/POP	5 / 11
C1 p.54	T	MOV/MOVC/MOVX, PUSH/POP	5 / 11
C1 p.55	T	MOV/MOVC/MOVX, PUSH/POP	5 / 11
C1 p.56	T	MOV/MOVC/MOVX, PUSH/POP	5 / 11
C1 p.57	T	MOV/MOVC/MOVX, PUSH/POP	5 / 11
C1 p.58	T	MOV/MOVC/MOVX, PUSH/POP	5 / 11
C1 p.59	T	MOV/MOVC/MOVX, PUSH/POP	5 / 11
C1 p.60	T	MOV/MOVC/MOVX, PUSH/POP	5 / 11
C1 p.61	T	XCH/XCHD exchange	13 / 51
C1 p.62	T	XCH/XCHD exchange	13 / 51
C1 p.63	T	Arithmetic forms and flags	6 / 14
C1 p.64	I	Arithmetic forms and flags	6 / 14
C1 p.65	T	Arithmetic forms and flags	6 / 14
C1 p.66	T	Arithmetic forms and flags	6 / 14
C1 p.67	T	Arithmetic forms and flags	6 / 14
C1 p.68	T	Arithmetic forms and flags	6 / 14

Source page	Kind	Recovered topic	Destination / page
C1 p.69	T	Arithmetic forms and flags	6 / 14
C1 p.70	T	Arithmetic forms and flags	6 / 14
C1 p.71	T	Arithmetic forms and flags	6 / 14
C1 p.72	T	Arithmetic forms and flags	6 / 14
C1 p.73	T	Logical, rotate and nibble forms	13 / 51
C1 p.74	I	Logical, rotate and nibble forms	13 / 51
C1 p.75	I	Logical, rotate and nibble forms	13 / 51
C1 p.76	T	Logical, rotate and nibble forms	13 / 51
C1 p.77	T	Logical, rotate and nibble forms	13 / 51
C1 p.78	T	Logical, rotate and nibble forms	13 / 51
C1 p.79	T	Logical, rotate and nibble forms	13 / 51
C1 p.80	T	Logical, rotate and nibble forms	13 / 51
C1 p.81	T	Logical, rotate and nibble forms	13 / 51
C1 p.82	T	Logical, rotate and nibble forms	13 / 51
C1 p.83	T	Logical, rotate and nibble forms	13 / 51
C1 p.84	T	Logical, rotate and nibble forms	13 / 51
C1 p.85	T	Bit/Boolean instruction tables	13 / 51
C1 p.86	I	Bit/Boolean instruction tables	13 / 51
C1 p.87	I	Bit/Boolean instruction tables	13 / 51
C1 p.88	I	Bit/Boolean instruction tables	13 / 51
C1 p.89	I	Bit/Boolean instruction tables	13 / 51
C1 p.90	T	Bit/Boolean instruction tables	13 / 51
C1 p.91	T	Bit/Boolean instruction tables	13 / 51
C1 p.92	T	Bit/Boolean instruction tables	13 / 51
C1 p.93	T	Bit/Boolean instruction tables	13 / 51
C1 p.94	T	Bit/Boolean instruction tables	13 / 51
C1 p.95	T	Jumps, calls, conditional branches	7 / 20
C1 p.96	I	Jumps, calls, conditional branches	7 / 20
C1 p.97	I	Jumps, calls, conditional branches	7 / 20
C1 p.98	T	Jumps, calls, conditional branches	7 / 20
C1 p.99	I	Jumps, calls, conditional branches	7 / 20
C1 p.100	T	Jumps, calls, conditional branches	7 / 20
C1 p.101	T	Jumps, calls, conditional branches	7 / 20
C1 p.102	T	Jumps, calls, conditional branches	7 / 20
C1 p.103	T	Jumps, calls, conditional branches	7 / 20
C1 p.104	T	Jumps, calls, conditional branches	7 / 20
C1 p.105	T	Jumps, calls, conditional branches	7 / 20
C1 p.106	T	Jumps, calls, conditional branches	7 / 20
C1 p.107	T	Jumps, calls, conditional branches	7 / 20
C1 p.108	T	Jumps, calls, conditional branches	7 / 20
C1 p.109	T	Jumps, calls, conditional branches	7 / 20
C1 p.110	T	Jumps, calls, conditional branches	7 / 20
C1 p.111	T	Jumps, calls, conditional branches	7 / 20
C1 p.112	T	Jumps, calls, conditional branches	7 / 20
C1 p.113	T	Instruction-set summary	12 / 49
C2 p.1	T	Deck cover	1 / 4
C2 p.2	I	Listing and assembly workflow	3 / 7
C2 p.3	I	25H+34H+12H and code-memory bytes	6.3 / 15

Source page	Kind	Recovered topic	Destination / page
C2 p.4	I	25H+34H+12H and code-memory bytes	6.3 / 15
C2 p.5	I	PSW and bank selection diagrams	6 / 14
C2 p.6	I	PSW and bank selection diagrams	6 / 14
C2 p.7	I	PSW and bank selection diagrams	6 / 14
C2 p.8	I	Addition with external result/carry	6.4 / 16
C2 p.9	I	Multiplication, two-byte product	6.8 / 18
C2 p.10	I	Division, quotient/remainder/base	6.9 / 18
C2 p.11	I	Five-byte block exchange	5.5 / 13
C2 p.12	I	Inclusive 17-byte block transfer	5.4 / 12
C3 p.1	T	Long/short jump forms and range	7 / 20
C3 p.2	T	Long/short jump forms and range	7 / 20
C3 p.3	T	Long/short jump forms and range	7 / 20
C3 p.4	T	Long/short jump forms and range	7 / 20
C3 p.5	T	Long/short jump forms and range	7 / 20
C3 p.6	T	Long/short jump forms and range	7 / 20
C3 p.7	T	Conditional jump and DJNZ	7 / 20
C3 p.8	I	Conditional jump and DJNZ	7 / 20
C3 p.9	T	Conditional jump and DJNZ	7 / 20
C3 p.10	T	ACALL/LCALL and subroutines	7 / 20
C3 p.11	T	ACALL/LCALL and subroutines	7 / 20
C3 p.12	T	ACALL/LCALL and subroutines	7 / 20
C3 p.13	T	Call/delay example and END layout	7.6 / 22
C3 p.14	T	Call/delay example and END layout	7.6 / 22
C4 p.1	T	Deck cover and syllabus	1 / 4
C4 p.2	I	Deck cover and syllabus	1 / 4
C4 p.3	I	Deck cover and syllabus	1 / 4
C4 p.4	I	Port architecture/input release	8.5 / 25
C4 p.5	M	Port architecture/input release	8.5 / 25
C4 p.6	M	Port architecture/input release	8.5 / 25
C4 p.7	I	55H/AAH output pattern	7.7 / 23
C4 p.8	T	Whole-port toggle and counted delay	7.6 / 22
C4 p.9	T	Whole-port toggle and counted delay	7.6 / 22
C4 p.10	I	Whole-port toggle and counted delay	7.6 / 22
C4 p.11	I	Bit-level output and instructions	13 / 51
C4 p.12	I	Bit-level output and instructions	13 / 51
C4 p.13	I	Bit-level output and instructions	13 / 51
C4 p.14	I	P1 input to P2 copy	8.8 / 28
C4 p.15	I	Switch/data/strobe input example	7.8 / 23
C4 p.16	I	Oven/pulse output signal boundary	8.5 / 25
C5 p.1	T	Deck cover and syllabus	1 / 4
C5 p.2	I	Deck cover and syllabus	1 / 4
C5 p.3	I	Deck cover and syllabus	1 / 4
C5 p.4	M	Assembly versus C workflow	3 / 7
C5 p.5	M	Assembly versus C workflow	3 / 7
C5 p.6	I	C51 integer/bit/SFR data types	8 / 24
C5 p.7	I	C51 integer/bit/SFR data types	8 / 24
C5 p.8	M	C51 integer/bit/SFR data types	8 / 24
C5 p.9	M	C51 integer/bit/SFR data types	8 / 24

Source page	Kind	Recovered topic	Destination / page
C5 p.10	M	C51 integer/bit/SFR data types	8 / 24
C5 p.11	M	C51 integer/bit/SFR data types	8 / 24
C5 p.12	I	C51 integer/bit/SFR data types	8 / 24
C5 p.13	I	Alternating 55H/AAH, delay helper	8.6 / 26
C5 p.14	I	ASCII ten-character bounds	8.10 / 29
C5 p.15	I	Delayed P1 sample to P2	8.8 / 28
C5 p.16	I	P0 conditional output routing	8.9 / 29
C5 p.17	I	One-bit toggle/delay	8.7 / 27
C5 p.18	I	Sensor-controlled buzzer signal	8.12 / 30
C5 p.19	I	LCD data/strobe scope boundary	13 / 51
C5 p.20	I	Boolean input inversion	8.11 / 30
C5 p.21	I	C bitwise and Boolean operators	8 / 24
C6 p.1	I	Deck cover	1 / 4
C6 p.2	T	Timer registers and widths	9 / 31
C6 p.3	T	Timer registers and widths	9 / 31
C6 p.4	T	TMOD fields/clock/source/gate	9 / 31
C6 p.5	M	TMOD fields/clock/source/gate	9 / 31
C6 p.6	M	TMOD fields/clock/source/gate	9 / 31
C6 p.7	M	TMOD fields/clock/source/gate	9 / 31
C6 p.8	T	Mode 1 count/control procedure	9.2 / 32
C6 p.9	T	Mode 1 count/control procedure	9.2 / 32
C6 p.10	T	FFF2H waveform and overlap	9.3 / 32
C6 p.11	T	FFF2H waveform and overlap	9.3 / 32
C6 p.12	T	FFF2H waveform and overlap	9.3 / 32
C6 p.13	M	FFF2H waveform and overlap	9.3 / 32
C6 p.14	T	FFF2H waveform and overlap	9.3 / 32
C6 p.15	T	B83EH delay calculations	9.3 / 32
C6 p.16	T	7634H waveform calculation	9.3 / 32
C6 p.17	T	7634H waveform calculation	9.3 / 32
C6 p.18	T	5 ms pulse and EE00H	9.3 / 32
C6 p.19	T	5 ms pulse and EE00H	9.3 / 32
C6 p.20	T	FF1AH and timer mismatch	9.3 / 32
C6 p.21	T	50 Hz/DC00H correction	10.3 / 37
C6 p.22	T	0108H repeated overflow	9.3 / 32
C6 p.23	T	Mode 2 reload/initialization	9.2 / 32
C6 p.24	T	Mode 2 reload/initialization	9.2 / 32
C6 p.25	T	Mode 2 reload/initialization	9.2 / 32
C6 p.26	T	TH1=5 first/steady intervals	9.3 / 32
C6 p.27	T	256 ticks*250 and 7.2 Hz	9.3 / 32
C6 p.28	M	Byte representation of negative reload	9.3 / 32
C6 p.29	T	Counter source and pin selection	10.7 / 39
C6 p.30	T	Counter source and pin selection	10.7 / 39
C6 p.31	T	Counter source and pin selection	10.7 / 39
C6 p.32	T	Counter source and pin selection	10.7 / 39
C6 p.33	M	TCON bit diagram/control	9 / 31
C6 p.34	T	TCON bit diagram/control	9 / 31
C6 p.35	T	TCON bit diagram/control	9 / 31
C6 p.36	M	C timer registers and clock assumptions	9.3 / 32

Source page	Kind	Recovered topic	Destination / page
C6 p.37	T	C timer registers and clock assumptions	9.3 / 32
C6 p.38	M	4BFDH nominal 50 ms program	10.1 / 35
C6 p.39	M	A5FEH repeated 25 ms program	10.2 / 36
C6 p.40	T	Switch 500/750 Hz and pin mismatch	10.6 / 39
C6 p.41	M	Switch 500/750 Hz and pin mismatch	10.6 / 39
C6 p.42	M	Mode 2 long delay/loop compensation	10.5 / 38
C6 p.43	M	Mode 2 nominal 2.5kHz	10.4 / 37
C6 p.44	T	Mode 2 external 8-bit counter	10.8 / 40
C6 p.45	T	Mode 1 external 16-bit counter	10.9 / 41
C6 p.46	I	Serial divider outside Module 2	13 / 51
C7 p.1	T	Deck cover	1 / 4
C7 p.2	T	Timer concepts/register overview	9 / 31
C7 p.3	I	Timer concepts/register overview	9 / 31
C7 p.4	T	Timer concepts/register overview	9 / 31
C7 p.5	T	TMOD/TCON/TH/TL diagrams	9 / 31
C7 p.6	T	TMOD/TCON/TH/TL diagrams	9 / 31
C7 p.7	T	TMOD/TCON/TH/TL diagrams	9 / 31
C7 p.8	I	TMOD/TCON/TH/TL diagrams	9 / 31
C7 p.9	T	Source/GATE and modes0/1/2	9.2 / 32
C7 p.10	T	Source/GATE and modes0/1/2	9.2 / 32
C7 p.11	I	Source/GATE and modes0/1/2	9.2 / 32
C7 p.12	T	Source/GATE and modes0/1/2	9.2 / 32
C7 p.13	T	Source/GATE and modes0/1/2	9.2 / 32
C7 p.14	I	Source/GATE and modes0/1/2	9.2 / 32
C7 p.15	I	Mode 3 supporting context	9.2 / 32
C7 p.16	T	Mode 1 programming procedure	10 / 35
C7 p.17	M	Mode 1 count and timing diagrams	9.3 / 32
C7 p.18	I	Mode 1 count and timing diagrams	9.3 / 32
C7 p.19	I	Mode 1 count and timing diagrams	9.3 / 32
C7 p.20	I	Mode 1 count and timing diagrams	9.3 / 32
C7 p.21	M	Timer 1 waveform/TH-TL values	9.3 / 32
C7 p.22	T	Tick units/clock examples	9.3 / 32
C7 p.23	T	50 Hz derived preload and waveform	10.3 / 37
C7 p.24	I	50 Hz derived preload and waveform	10.3 / 37
C7 p.25	T	Delay-to-preload method	9.3 / 32
C7 p.26	T	35000 ticks/7748H calculation	9.3 / 32
C7 p.27	I	Nested/repeated delay example	9.3 / 32
C7 p.28	T	Nested/repeated delay example	9.3 / 32
C7 p.29	T	Mode 2 auto-reload operation	9.2 / 32
C7 p.30	T	Mode 2 auto-reload operation	9.2 / 32
C7 p.31	M	Mode 2 minimum ideal frequency	9.3 / 32
C7 p.32	T	Byte truncation/negative reload	9.3 / 32
C7 p.33	T	Counter source/control	10.7 / 39
C7 p.34	T	Counter source/control	10.7 / 39
C7 p.35	I	Counter source/control	10.7 / 39
C7 p.36	M	Counter 1 mode 2 example/display	10.8 / 40
C7 p.37	I	Counter 1 mode 2 example/display	10.8 / 40
C7 p.38	I	Counter 1 mode 2 example/display	10.8 / 40

Source page	Kind	Recovered topic	Destination / page
C7 p.39	I	Counter 0 mode 1 example/display	10.9 / 41
C7 p.40	I	Counter 0 mode 1 example/display	10.9 / 41
C7 p.41	I	Counter 0 mode 1 example/display	10.9 / 41
C7 p.42	T	TCON and TR/TF control bits	9 / 31
C7 p.43	I	TCON and TR/TF control bits	9 / 31
C7 p.44	T	TCON and TR/TF control bits	9 / 31
C7 p.45	I	T0/T1 versus INT0/INT1 gates	10.7 / 39
C7 p.46	I	T0/T1 versus INT0/INT1 gates	10.7 / 39
C7 p.47	I	Closing slide	1 / 4

14.3 Primary references and access record

Accessed 7 October 2026. Links are topic-specific; exact device differences are respected. The slides supply classroom examples; sources below resolve architecture/syntax issues. Guide prose and answers are authored explanations, with no textbook passages reproduced.

R1. Intel, *MCS-51 Microcontroller Family User's Manual*, order 272383-002, February 1994, Intel-authored manual hosted by MIT. Architectural Overview / memory, ports, timers; Programmer's Guide / instruction definitions, stack and encoding. The accessible PDF was downloaded and searched locally, with relevant diagrams/pages inspected.
R2. Keil, 8051 opcode/length table and instruction definitions and conventions. Used to cross-check operand forms, encoded lengths and classic cycle counts.
R3. Keil, individual definitions: AJMP and page boundary, MOVC, ADDC. Broader arithmetic/control definitions cross-checked against R1/R8 rather than relying on OCR tables.
R4. Keil A51, DB directive and bit-symbol directive. Syntax/context checked with accessible assembler documentation; unused examples' generic metadata were not adopted.
R5. Keil C51, type representation note, sbit definition/linkage note. Some legacy C51 manual URLs redirected to an inaccessible migrated page; their unavailable bodies were not claimed as consulted. Hardware/type conventions also underwent translated compilation.
R6. Analog Devices / Maxim, Ultra-High-Speed Flash Microcontroller User's Guide, revision 11, November 2011: clock-control registers and Timer/Counters. Enhanced-device timing is used only for the DS89C420 distinction.
R7. Keil, integer promotion and complement, support article 2483. Used for byte-width/promotion pitfalls; Boolean logical inversion is explained separately.
R8. TEMIC / MATRA MHS, C51 Microcontroller Programming Manual,14 January 1997: classic-compatible memory/register description, TMOD/TCON, instruction summary and individual definitions. Its additional derivative registers do not change the original 8051 baseline used here.

Publisher bibliography/contents checks: Pearson India ECE catalogue; Mazidi second-edition contents in an international reprint; Cengage Ayala third-edition record and contents. Editions/reprints may paginate differently; chapter numbers/titles, not invented pages, are mapped in Section 12.3.

Local context: the official syllabus physical pp.128–131 controls scope/exam/books. Module 1 supplies prerequisite context; the separate lab manual supplies only relevant transfer/arithmetic practice. Anthropic.tex was inspected as a style template, not a factual source; its relative font dependency, generic metadata and Python listings were replaced. Module 3/4, PE/MOS typography was inspected; this guide uses available Rakuten Serif App real regular/bold/italic faces, Poppins headings and embedded DejaVu Sans Mono code.

14.4 Verification record and remaining limits

Content and calculations. All 269 source pages were inventoried, rendered and reviewed, including every named image-heavy page. Programs define all helpers/labels and explicitly distinguish hardware assumptions. Timer calculations use exact rational $12/11059200$ internally; counts/preloads and decimal results were recomputed independently. Flag examples were cross-checked with the architectural definitions and functional traces. The official syllabus coverage table resolves to final pages.

Translated assembly and compilation. All 17 complete A51-style ALPs assembled and linked after a narrow syntax translation to SDCC's `sdas8051/sdld`; all 16 complete C51-style programs compiled without warnings using SDCC4.2.0, MCS51 small model, C99 and speed optimisation. Assembly translation replaces ORG/DB notation and H-suffix constants, supplies standard SFR addresses and removes source-only $MOD51/END. C translation replaces the header with 8051.h, maps sbit aliases to absolute bit declarations, and maps code/bit qualifiers. Algorithms and numeric values are unchanged. This establishes syntax/operand viability in the translated toolchain, *not* identical C51 machine code or native Keil compilation.

Functional simulation. uCsim0.6.4 classic 8051 model passed 23asserted cases: A01–A14 plus a zero-divisor variant; high/low inversion and both routing branches; C08 overflow-driven output and timer selection; C11 mode/reload/run setup; C14/C15 count-display logic via register injection. Checks included all 17 block-copy bytes, both 5-byte exchanged blocks, external product/sum/remainder bytes, saved borrow, BCD adjustment, and the low/high return-address bytes with restoredSP.

Limits. No native Keil license/compiler or physical board was used. Simulator startup/time behavior is specific to the translated SDCC image. External counter pin pulses did not advance the count through the simulator's attempted port-injection interface; the successful counter tests instead inject count registers and validate display logic. Physical edge sampling, switch bounce, electrical loading and oscillator tolerance remain untested. Hardware-timer formula results are ideal; the C waveforms have unmeasured polling/edge overhead and jitter. Full textbook bodies were unavailable; publisher bibliography/contents were checked. The division base and LCD-space recovery are explicitly labelled rather than silently guessed.

PDF and preservation checks. Both editions are generated from one source with only colour definitions changed. Final validation compares every page's extracted text and geometry, checks page counts, embedded fonts, links/bookmarks and resolved references, renders and visually reviews every page in both editions, and tests pure-black page/panel backgrounds. Original study files are hash-compared before delivery. Only the two final PDFs are added to the study directory; all build/OCR/download/simulation artifacts remain in temporary storage.